To calculate size of array.

int n = sizeof(array) / sizeof(array[0]).

In 2D array [R][C] Total elements = rows x col

To find col = size of(arr[0]) / size of(arr[0][0])

↓

first row / 1 element

rows = size of (arr) / size of (arr[0])

int **p A pointer storing address of another pointer

int *p = new int(20) p has stored a value of 20.

new int[5] → creates array of 5 int.

int * fun() {

int a = 20

return &a

}

→ Pointer function that returns add

'a' only exist while fun() runs as soon as retn is executed Fun() is destroyed. &a → dangling ptr

Better way! dynamically

int * fun() {

return int new (23);

}

Pass by val

```
Void fn(int *p)
{
    p: new int[10];
}
```

In main

```
int * p= nullptr
cout<< *p;
```

outputs 10

Pass by ref

```
void fn (int &p)
```

⋮

outputs 10 —

ADT → hidden implementation of how data is organized and what op are allowed/ can be performed.

↳ Stack
(Stores data in LIFO order)

Push: 10 , 20 , 30



Pop → 30

Queue (FIFO)

Enqueue 10 20 30

Front → 10

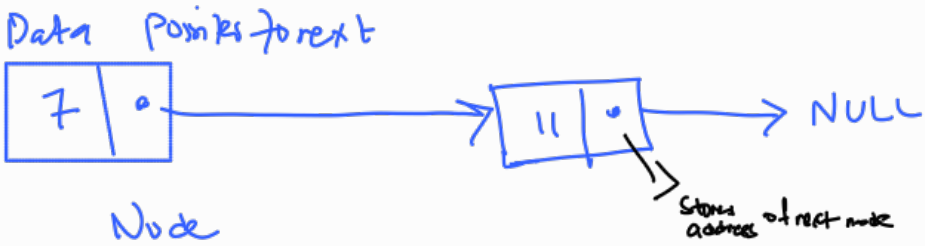
ADT → more kind of
store data wrestm
organize it and
what operations
we can apply
(NOT the method used
to organize data)

ADT is purely logical Blueprint and defines what should happen but has no body only operations

e.g Smartphone ADT
makeCall(), SendMessage()

ADT is interface (Buttons you press) and
DS is implementation such
as LL, Stack

Linked list:



Benifit is that in array we cannot change its size after we created it. For that we need to create a new array and copy the elements of the previous array to the new. But with LL we can add a new node

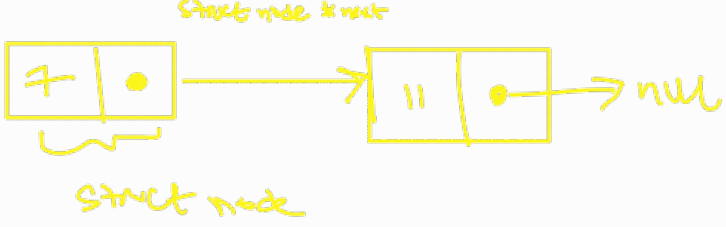
easier to insert in LL Rather than shifting in arr.

(LL are not continuous but still linked together) ←

1) LL → Takes more memory ⭐ due to pointers

2) Random Access in LL is not allowed as not continuous memory location.

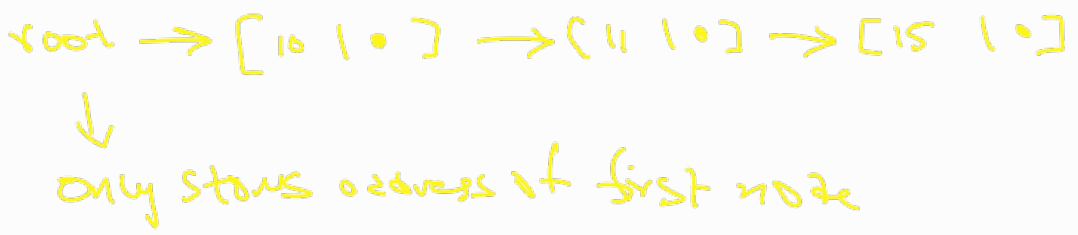
Implementation of LL



if(!head) → means head is null.
return nullptr;
if true mean LL is empty

```
Struct node {
    int data;          actual data
    Struct node *next;   address of next node
};

node * root;  Starting pointer  not a node
root = 0;
```

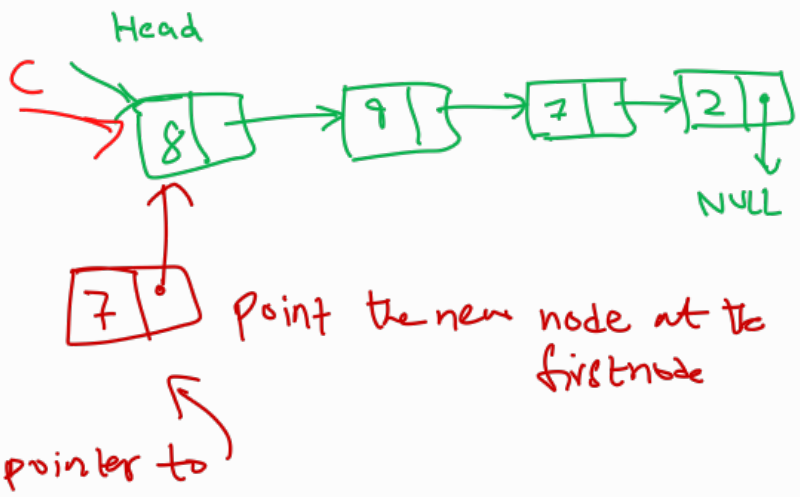


if root = null it means List is empty

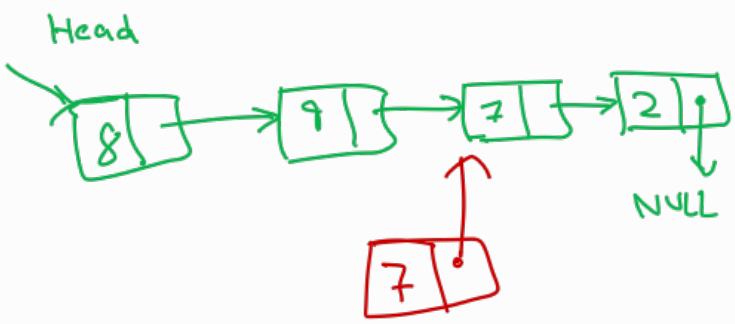
(Traversal of LL)

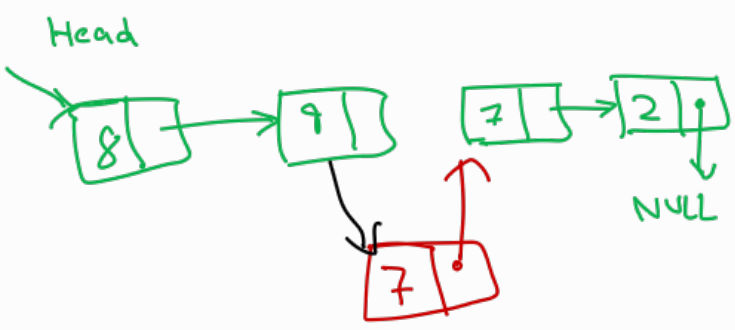
Insertion:

case1 insert at Beginning

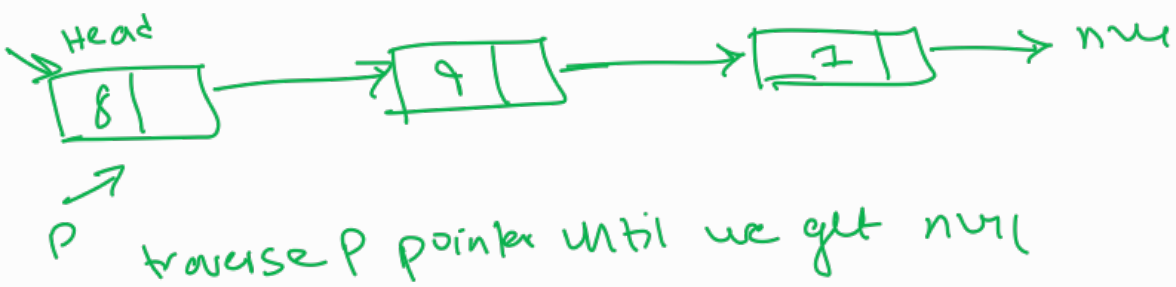


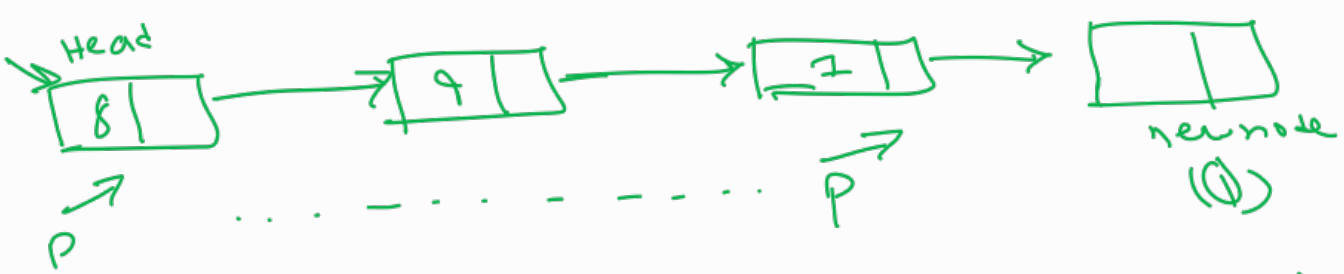
case 2: insert in between





case3 insert at end



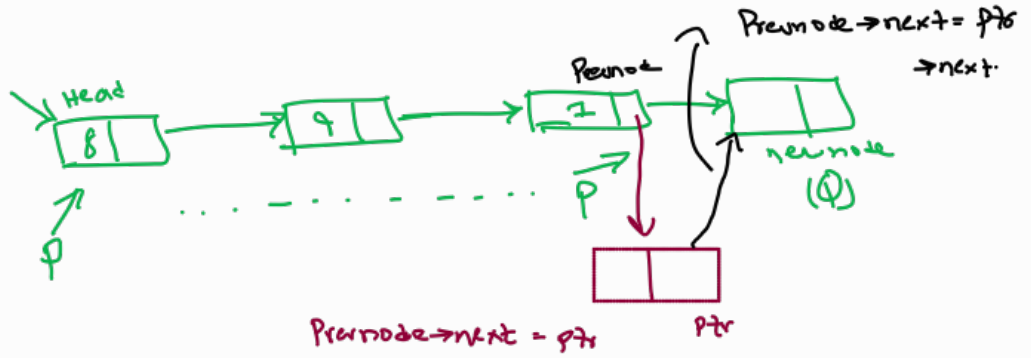


p->next = newnode;

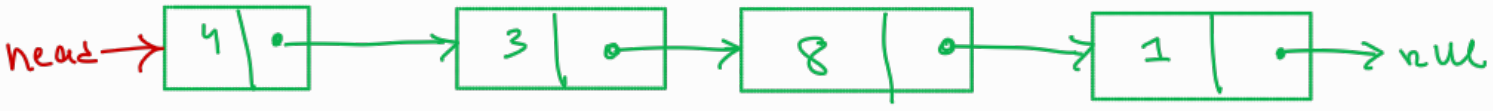
newnode->next = null;

case 4: Insert after a node → Same Logic as for insert in b/w 2 nodes

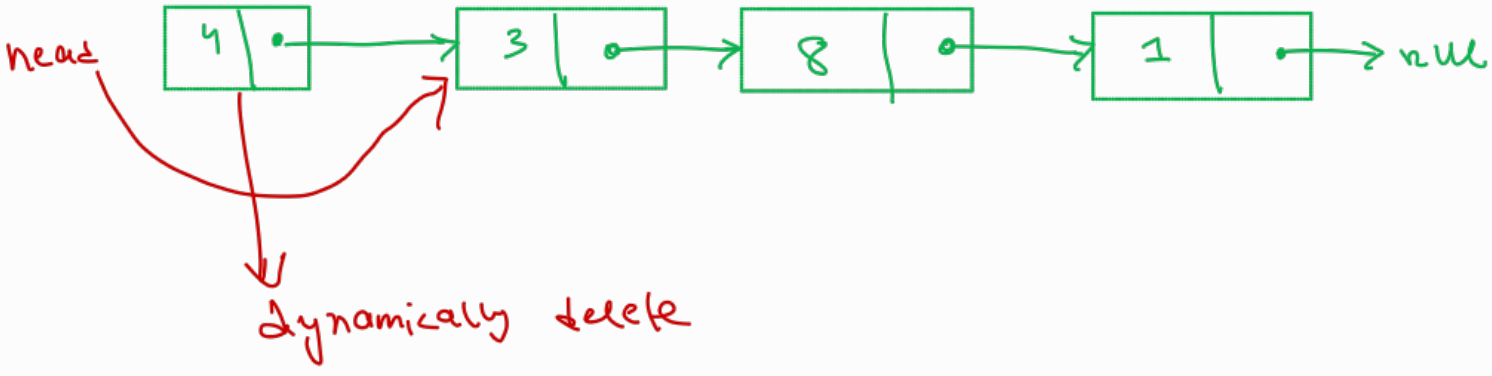
✱ For Singly LL.



(Deletion in Linked list)



cases :: Deleting first node

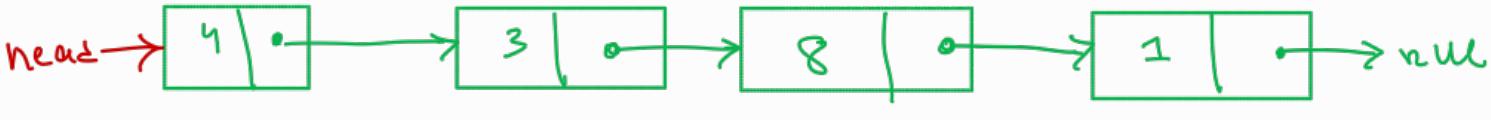


node * ptr = head;

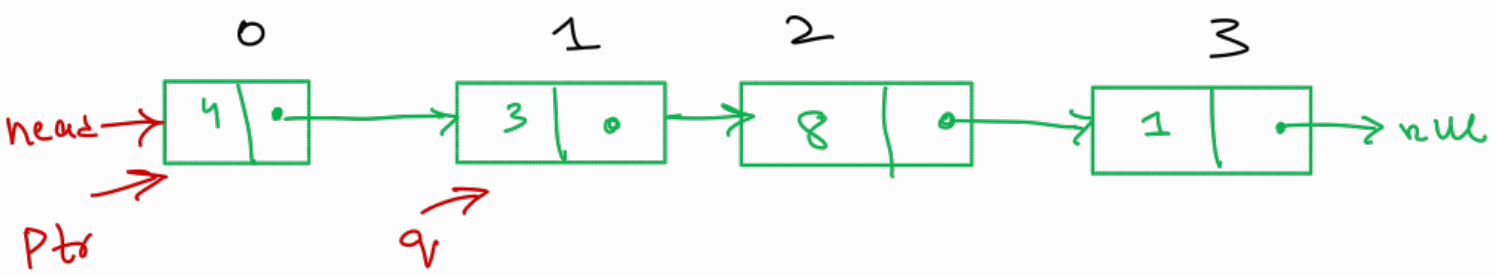
head = head → next;

delete ptr; → deleted first node memory.

case 2:: Deleting a node at a given index.



↑ delete this




```
Node *Ptr = head;
Node * q = head->next;
```

← we create two pointers ptr and q.

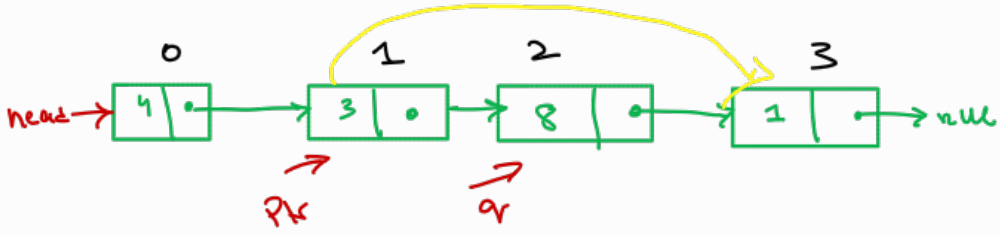
```
for(int i=0; i<index-1; i++)
{
```

index-1

lets say delete at 2.

```
ptr = ptr->next;
q = q->next;
```

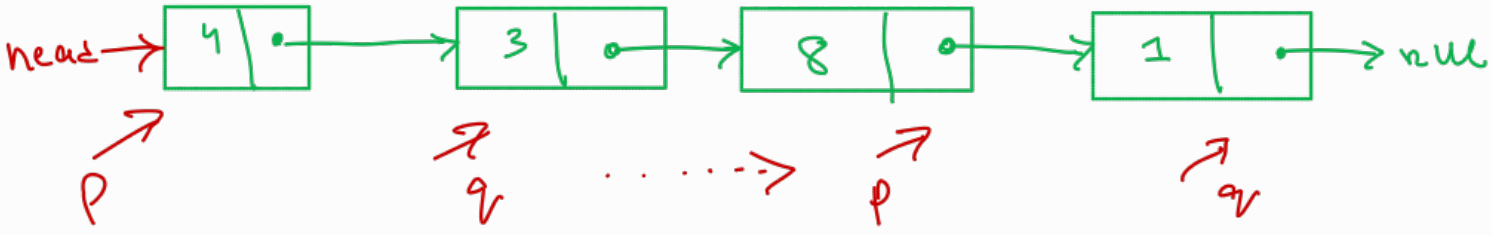
3



after loop exe...

```
ptr->next = q->next
delete q
```

Delete last node



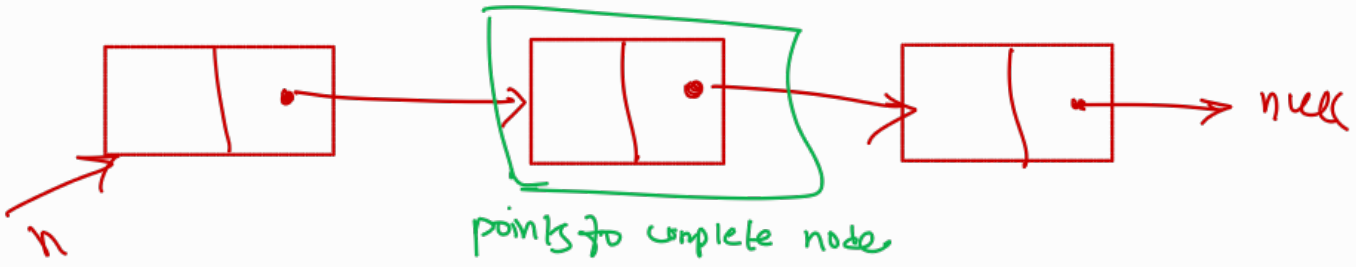

```
p->next = null
delete q;
```

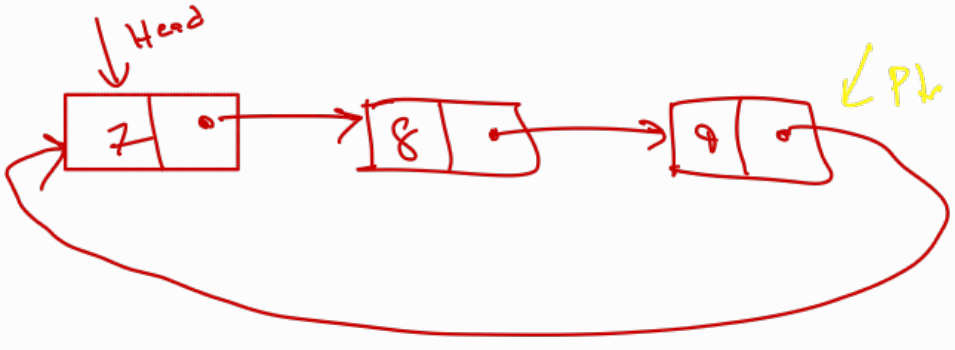
```
while(q->next != null)
{
    p = p->next
    q = q->next
}
p->next = null
delete q;
```

delete a node with a given value

Laptop code

(circular linked list)





Last node points to first node instead of null.

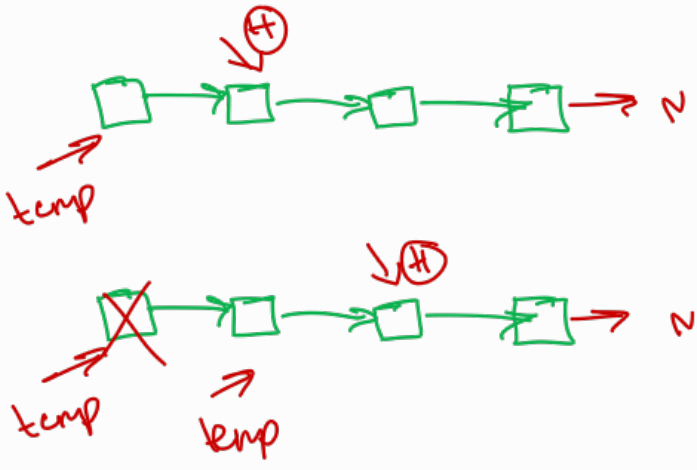
Ptr = head;

Printing / traversing circular LL

```
while (ptr->next != head)
{ cout<< ptr->data;      -> 8, 9
  ptr = ptr->next;
}
print (ptr->data)        -> 7
```

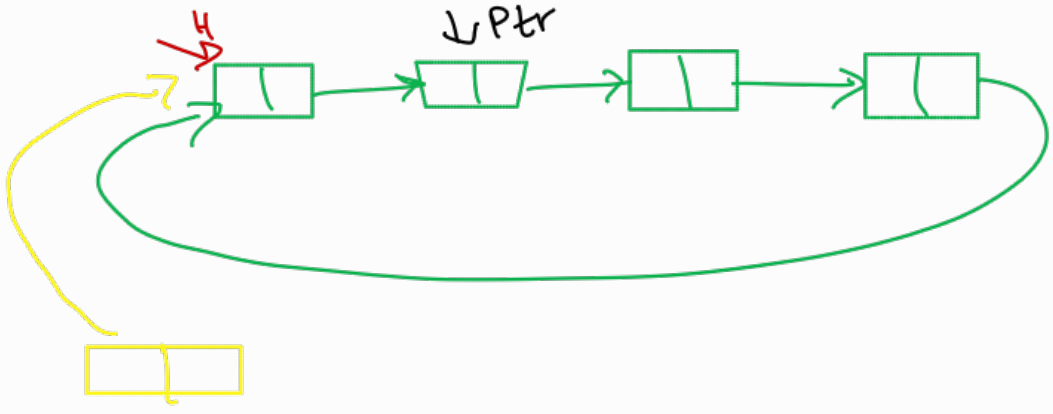
To empty a Singly LL Head → null

```
while (head != null)
{
    node * temp = head;
    head = head -> next;
    del temp
}
```



move head forward and delete the node behind it.

(Insertion)



insert here and also make it the head.

finding middle of array.

1 → 2 → 3 → 4 → 5

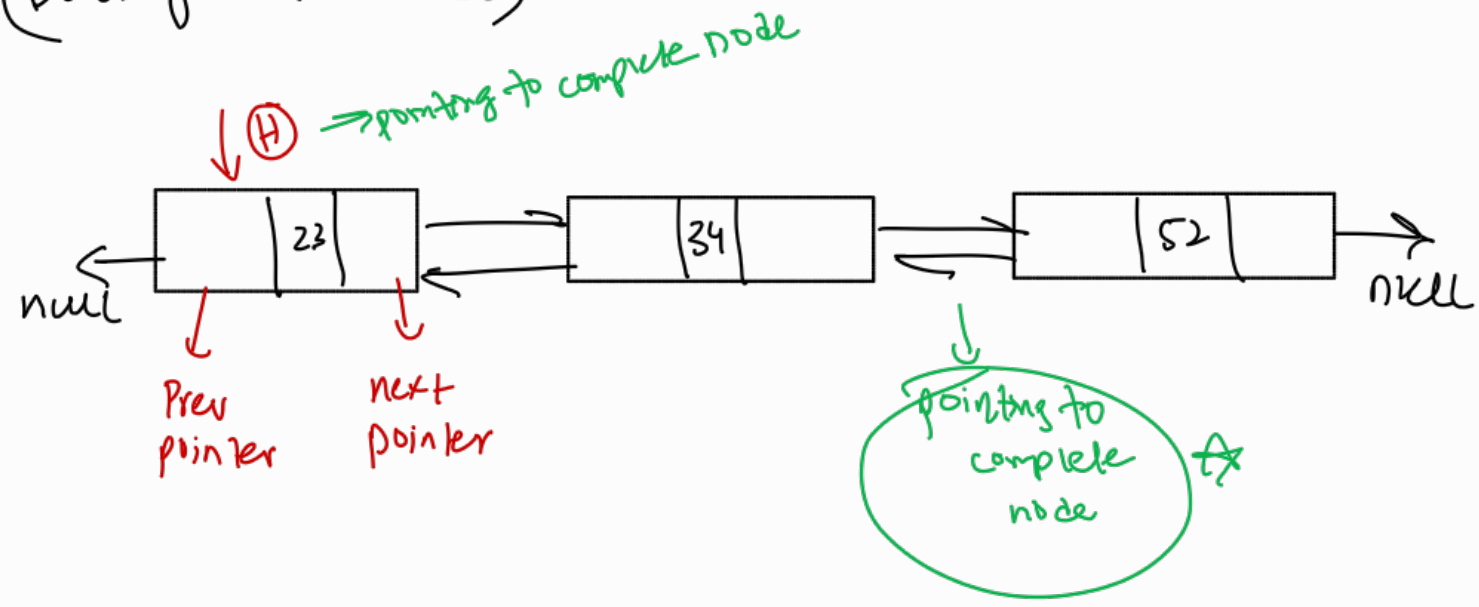
S,F ↓ odd nodes

Slow → 1 skip
fast → 2 skip

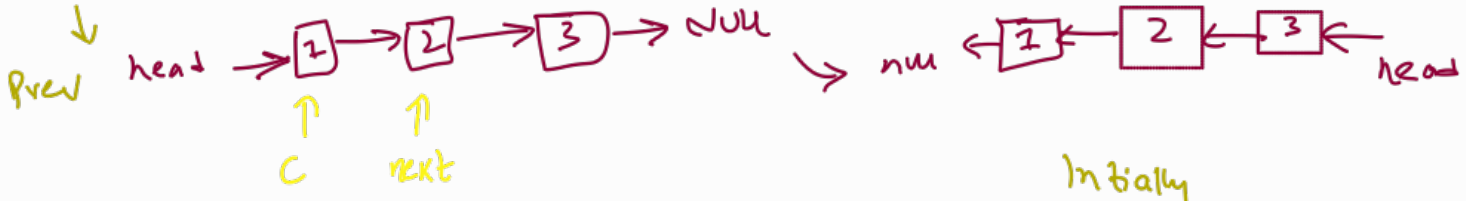
when fast is at last node → slow will always be at the middle

1	2	3	4	5	NULL
		S		F	

(Doubly linked list) ⇐

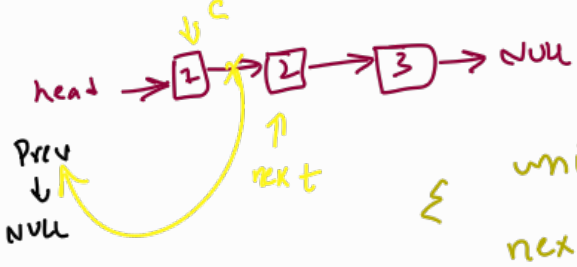


Reverse a LL

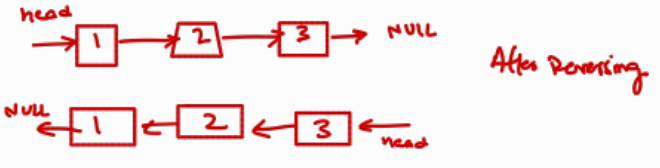


Initially
Prev = NULL
C = head
next = NULL

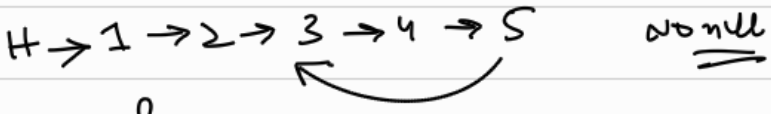
(Reverse connections of LL)




```
while (current != NULL)
{
    next = current -> next;
    current -> next = prev;
    Prev = current;
    current = next;
}
return prev → Becomes our new head.
```

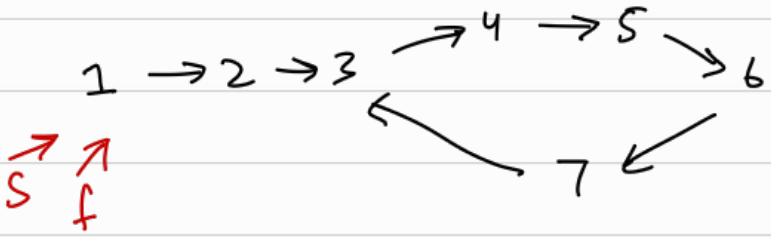


cycle detection.



no null

using slow | fast pointers

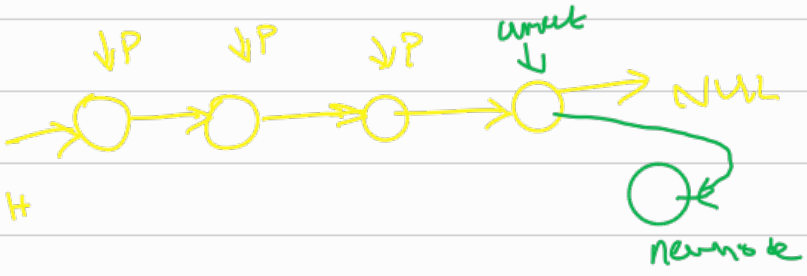


Slow p → moves 1 step.

fastp → moves 2 steps.

if fast pointer reaches null then no cycle is present.

if fast = slow then cycle is present.



current→nextnode = newnode

means current pointer will point to newnode

original list:

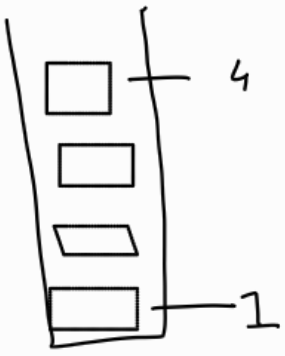
head = fun(head, 3)

1→2→3→4→5→6→7→8→9

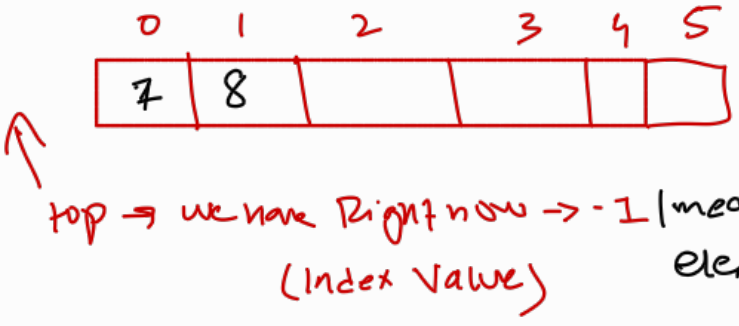


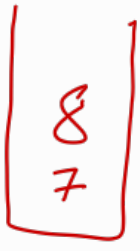
(Stacks) → linear data Structure

Last in first in



Implementing Stack with Array.





Steps to implement Stack using arrays.

1) fixed size array creation
2) top element.

```
Struct Stack {
    int Size;
    int top;
    int *arr;   → pointer
}
```

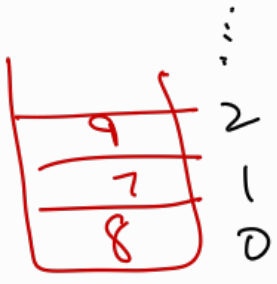
```
Struct Stack S;
  S.Size = 80;
  S.top = -1;   → means no element in array
  S.arr = new int [S.Size x Sizeof(int)];
                          ↓
                    Integer array
```

```
Struct Stack *S;   → Stack pointer use to pass in functions.
S→Size = 80;
S→top = -1;
S→arr = new int [S→Size * Size of (int)];
```

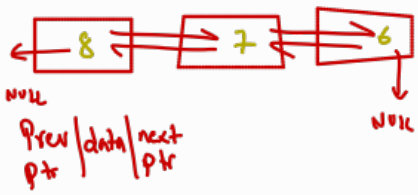
condition to check if Stack is empty or not.

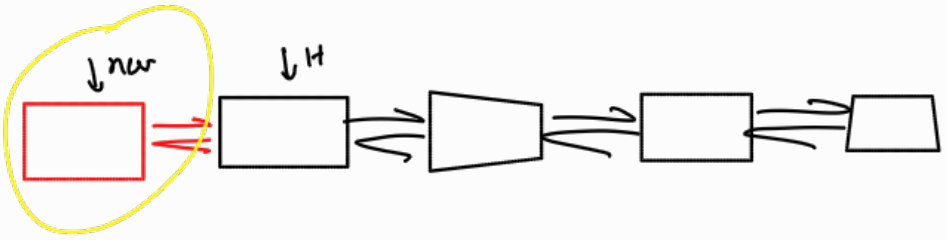
```
isempty (Struct Stack *ptr)
        if (ptr -> top == -1)
            return 1
```

isfull `if (ptr -> top == ptr -> Size - 1)`



Doubly linked list Insertion at start:



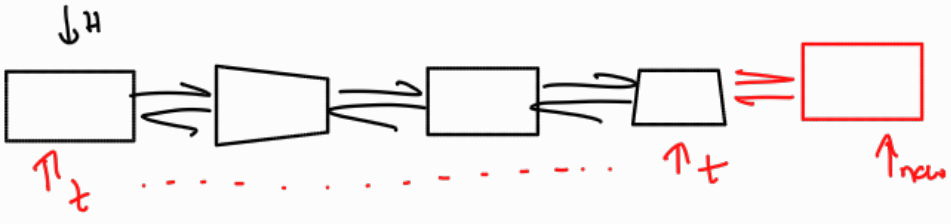


new → next = head

head → prev = new

head = new; → head now points to new

Inserting at the last:



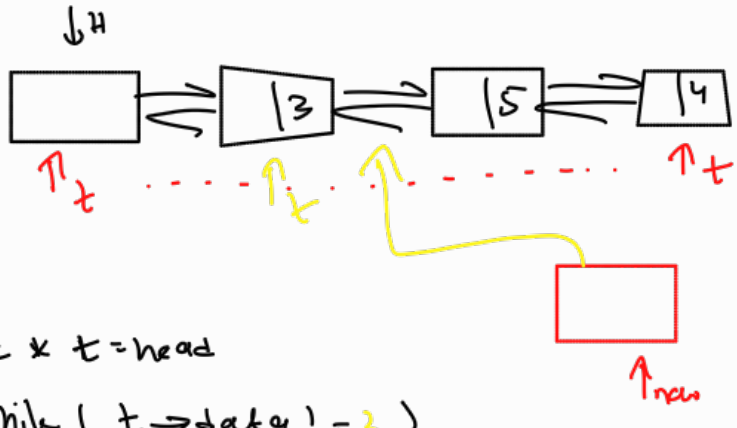
node * t = H;

while (t → next != NULL)

t → next = new

new → prev = t

Inserting after a node:



node * t = head

while (t → data != 3)

{ t = t → next;

}

t = new → prev

↓

this means we are just pointing t to new → prev

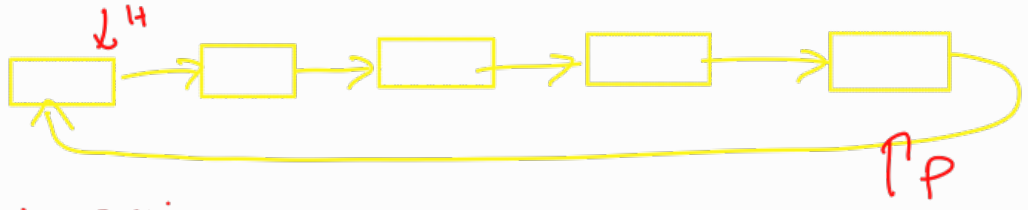
new → next = t → next

new → prev = t

t → next = new

new → next → prev = new

Circular L.L



Insert at starting



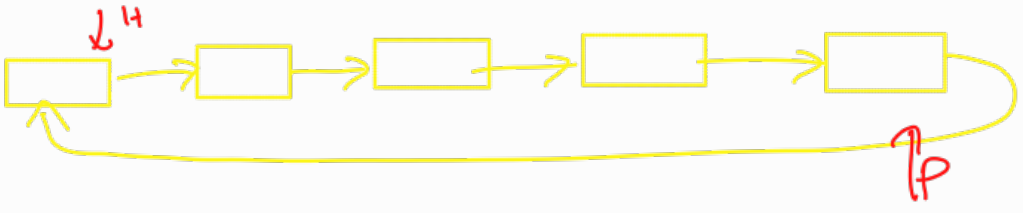
Traversal

```
Node * ptr = head;
while (ptr->next != head)
{
  print (ptr->data);
  ptr = ptr->next;
}
print (ptr->data);
```

```
node * p = head;
while (p->next != head)
{
    p = p->next;
}
p->next = new
new->next = head;   // point new->next where head is pointing
```

head = new; ⟶ head will point to new

Inserting at last:

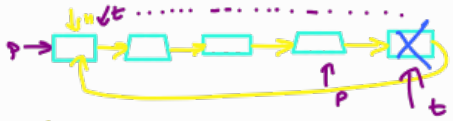





```
node *p = head;
while (p->next != head)
{
    p = p->next;
}
p->next = new
new->next = head
```

// new->next points to where head is pointing

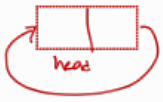
(circular LL deletion operation) (Deleting the last node)



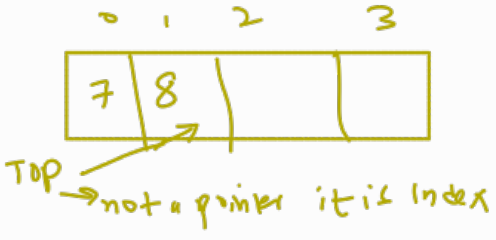
node *t = p;
t = head; p = NULL
while (t -> next != head)
{
p = t
t = t -> next // t moving 1 step forward as compared to p
}
p -> next = head
delete t

Empty circular LL

node * head = new node();
head -> next = head;



Implement Stack using array



Top = -1 no value in stack

→ fixed size array creation
→ Top element

```
Node Stack {
    int size;
    int top;
    int *arr;
};
```

```
In main
    Stack S;
    S.size = 80;
    S.top = -1;
    S.arr = new arr[S.size];
```



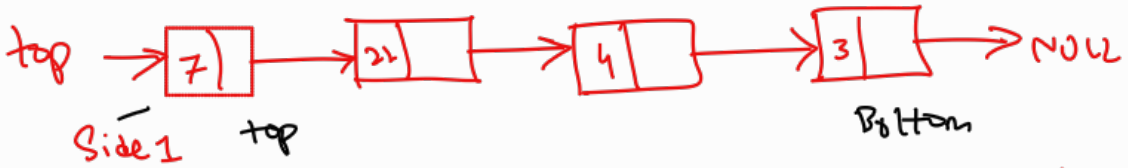
Push

```
if (isfull())
    cout<<"Stack overflow";
else
    Sp->top++;
    Sp->arr[Sp->top] = val;
```

Pop:

```
if (is empty)
    cout<<"Stack underflow";
    return -1;
}
else
    int val = Sp->arr[Sp->top];
    Sp->top = Sp->top - 1;
    return val;
```

Stack using LL.



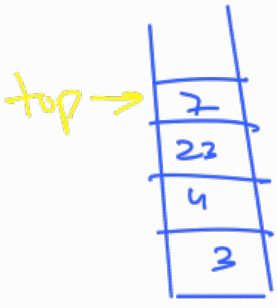
Side 1 will be use for push) pop operation

top

Bottom

class node {
int data
node * next;
}

→ Stack empty condition (top == NULL)
→ Stack is full Memory is exhausted



void IsEmpty (node * top)
if (top == NULL)
return 1;

void isfull
node * temp = new node();
if (temp = nullptr)
return true;
else
delete temp // allocation Succeded
return false
means Stack can accept more elements.

Push() // inserting node at index0 aka
{
node *temp = new node(13);
if (temp == NULL)
cout<< stack full;
else
temp->next = top;
top = temp;
↳ is the point pointing to first node

Pop() if (is empty) {
cout<< stack underflow
else
node * n = top;
top = top->next
int val = n->data;
delete n;
return val;

Infix a+b, a-b, P|q> x*5. operand <operator> operand.

Prefix <operator> operand2 operand2

+ab -pq

-xy *pb

Postfix

ab+ pq/

pqk

Infix A * (B+C) * D

Postfix ABC+ *D // Better for machine to understand.

Infix → postfix using stack



increasing precedence of operators

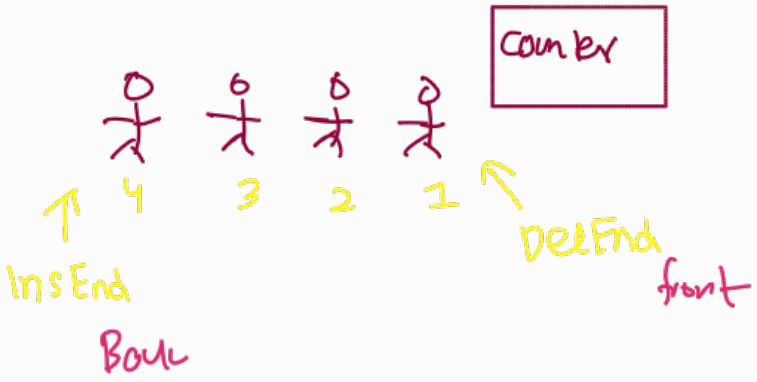
x - y/z - k * d

Push all operators first

xyz/-kd*-

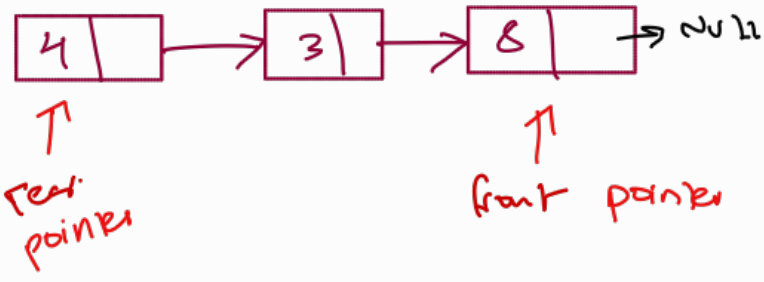
Queue → first come first serve

Things we need
1) Storage
2) Ins End
3) Del End



enqueue → insert
dequeue → delete

(Queue using LL)




```
Void enqueue (node* fwdptr, node* Rearptr, int val)
{
    node* newnode = new node(); //creates a new node
    if (newnode == NULL)
        cout<< "queue is full";
    else
        newnode->data = val;
        newnode->next = NULL
        if (fwdptr == Rearptr == NULL)        // if (queue is empty it means front and rear both are null
            fwdptr = Rearptr = newnode
        else
            Rearptr->next = newnode;
}
```

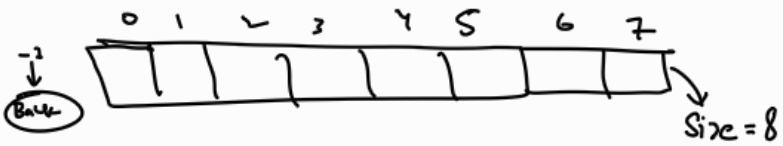
in queue we always add element at rear.

isempty queue
fwdptr == NULL

isfull queue
newnod = NULL

```
int dequeue // always happen from fwdptr.
    (node* fwdptr, int val)
{
    int val = -1;
    node* newptr = fwdptr;
// check for queue empty.
else
    fwdptr = fwdptr->next;
    val = newptr->data; // deletes the previous value
    free newptr;
    return val;
```

(Queue using Array based)



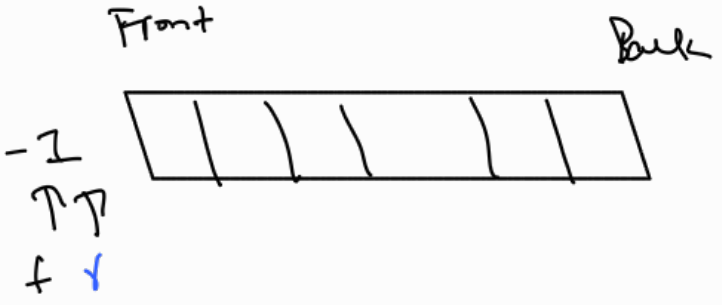
To insertin array
→ Increment back
→ insert the element

Remove



Remove: 7 element at 0.
Shift all elements.

Initally f, r Both pointing to -1.



Insert: Increment r and insert at r

Remove:
Remove element at f and Inc f.

```
Struct Queue {
  int size; →Size of array
  int f, > not pointers but
  int r; index values
  int *arr;
  };

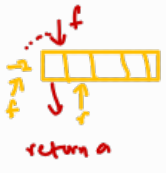
int main() {
  Struct Queue q;
  q.size = 10; // array size
  q.f = q.r = -1; // empty array Pos

  q.arr = new int [q.size];

Void enque (node * queue, int val)
  if (isfull(qr)) {
      cout << "overflow";
  else {
      q -> r = q -> r + 1;
    // q -> arr[q -> r] = val;
  insert val.
  }
```



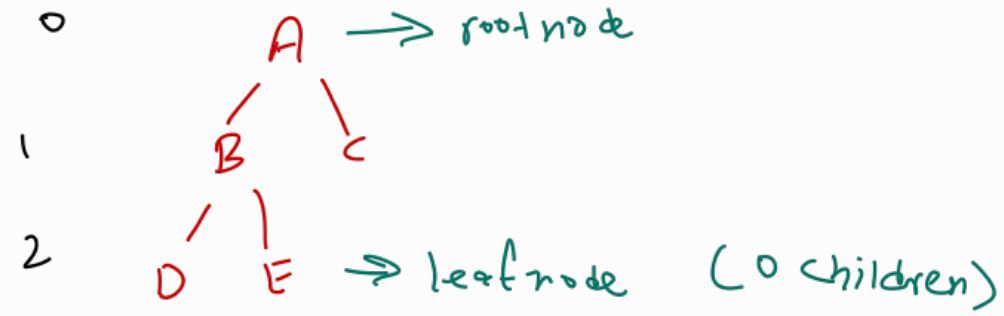

```
Int deque (node * queue) {
    Int a = -1
    if (q -> f == q -> r)
        cout << "queue is empty";
    else
        int a = arr [q -> f]
        q -> f = q -> f + 1
        return a;
```



Queue empty = f = -1
// full r = size - 1

if (q -> r == q -> size - 1)
return 1;

Tree



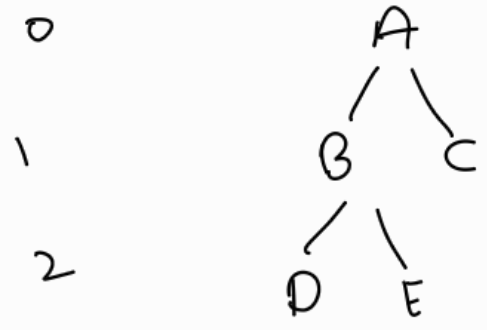
Degree of node = no of children

B → 2

Height of tree = Max level + 1

= 2 + 1 → 3

Binary → each node has atmost 2 children.



Max no of nodes

formula = $2^k - 1$

k = height

$2^3 - 1$

→ (7) nodes

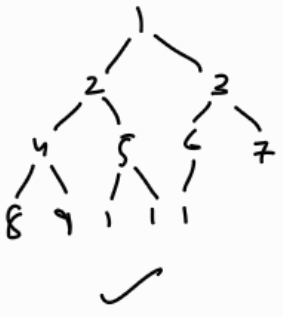
To complete Binary tree fill

Level by level

(L → R)

A complete Binary tree is when all levels are filled except the Last.

from L → R

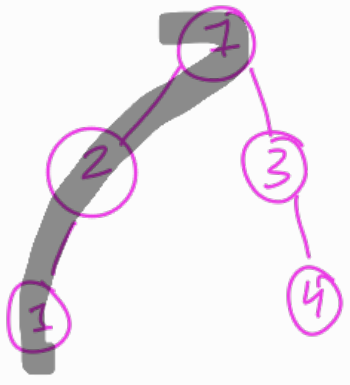


1) Preorder traversal

Root → left → Right subtree

Subtree

7 → 2 → 1 → 3 → 4



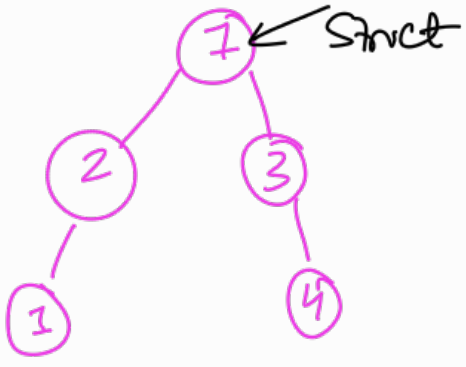
2) Postorder

leftsubtree → Rightsubtree → root

3) Inorder

leftSub → root → RightSub

Preorder:-




```
Void preorder (node * root) {
    if (root != null)
        {
          cout<< root -> data;
        preorder (root -> left)
          preorder (root -> right)

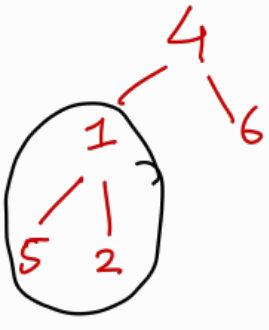
    }
```

Postorder

leftsubtree → rightsubtree → root

[ ] 6 4

5 1 2 6 4



Inorder leftSub Root RightSub

e.g



[5 1 2] 4 6

apply this for LeftSub tree



post order

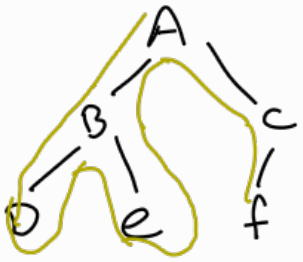
[4 11 5] 3 10

Preorder

10 [11 4 5] 3

Inorder

[4 11 5] 10 3



Preorder A B D e C f

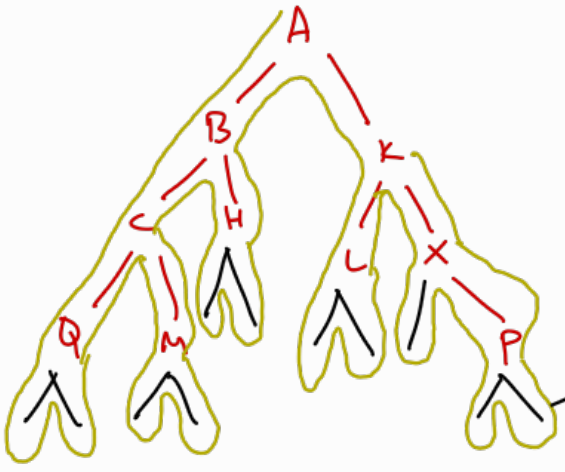
Inorder

Print only when v reach node for 2nd time

D B e A f C



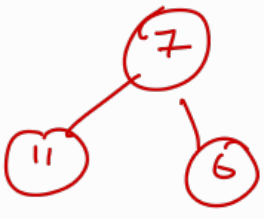
1) Preorder
2) Inorder
3) postorder



→ Make NULL △ at Leaf nodes for traversal

Binary Search tree

helps in fast Search

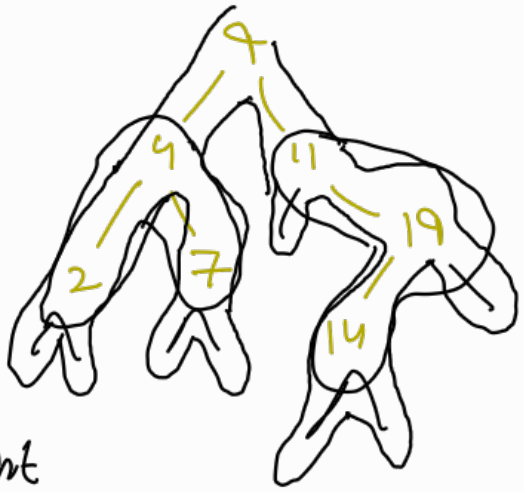


⇒ All nodes of leftsubtree < Rightsubtree

⇒ no duplicate nodes

→ To check

Inorder traversal must be in ascending order



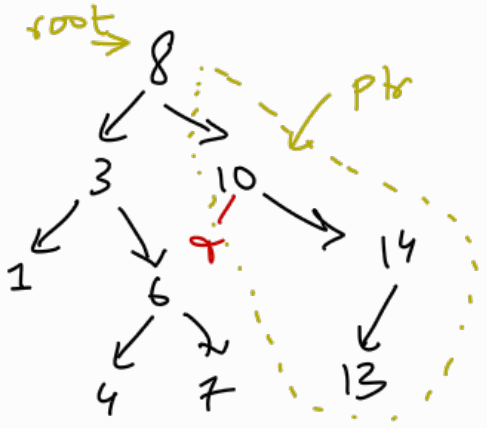
2 4 7 9 11 14 19 ✓

BST

left-root-Right

1) Insertion:

Searching 9.
Insert 9



9>8

→ search space limited

```
void insert ( node *root , int key)
{   node *prev = null  → used to remember the parent node
    node * ptr ;  → move throught the tree
    while (root != null)   → till we reach leaf node
    {
        Prev = root
        if (key == root → data)
                return ;
        else if (key < root→data)
                root = root → left;// go to left subtree
        else
                root = root → right; // go to Right subtree
    }
```

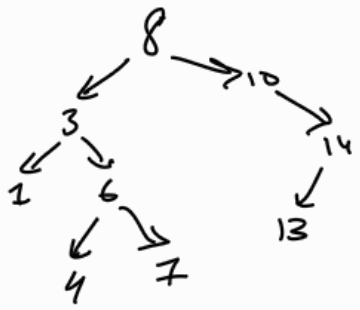
//now we are at null meaning reached correct empty space to insert.

```
nodenew = new node(key);
if (key < Prev → data)
    {   Prev → left = new
    }
else  Prev → right = new;
}
```



9<6

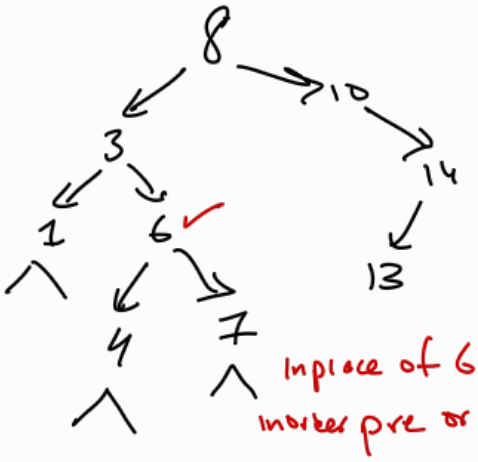
Deleting in BST



C1 → Leaf node

Search the node and then delete the node

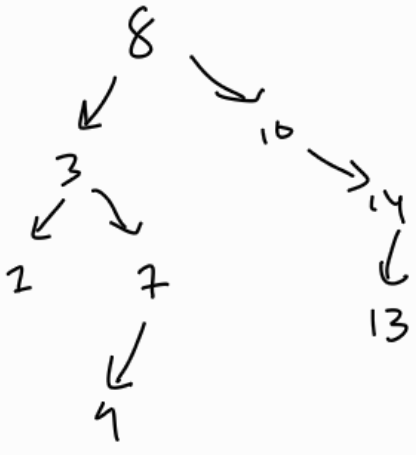
C2 node is non leaf node



1 3 4 6 7

either replace 6 by 4 or 7

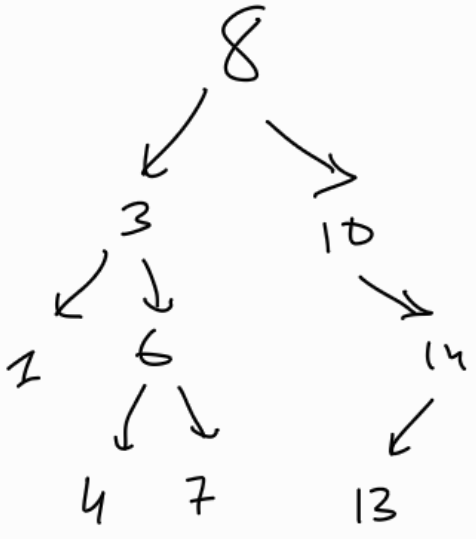
Inplace of 6 inorder pre or inorder post will come



e.g

C3 Root node

deleting 8

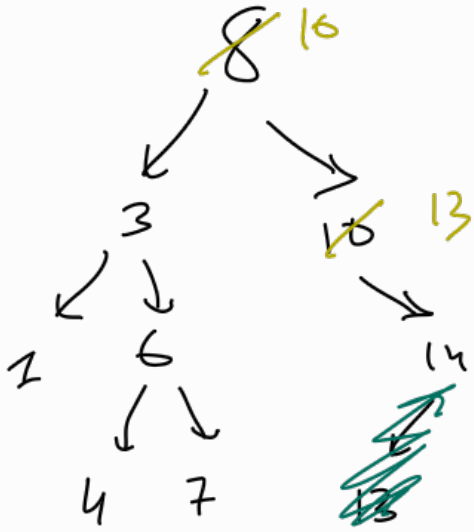


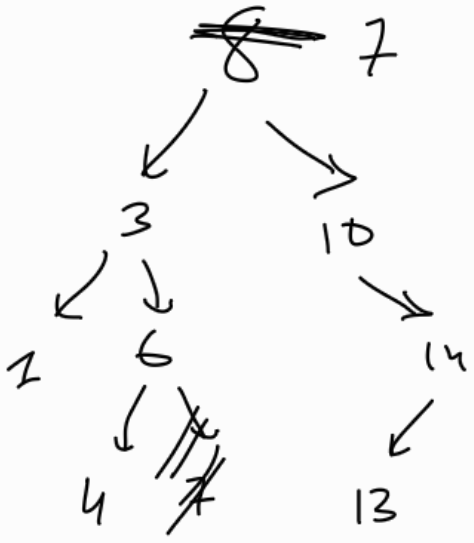
inorder → 1 3 4 6 7 8 10 13 14

Replace by either 7 or 10.

Preferably replace by 7 / leaf node

Inorder predecessor ↓ left subtree k right most child






```
Node * deleteNode (Node *root, int value)
{                                          // search for the node to be deleted
    if (value < root->data)
        deleteNode (root->left, value)
    else if (value > root->data)
        deleteNode (root->right, value)
    // else -> meaning we found the node
    else
        inOrderPre = inOrder (root)
        root->data = inOrderPre->data;
        deleteNode (root->leftPtr, inOrderPre)   // deleting the inorder predecessor
```

```
Struct node* inOrder (root)
{
  root = root->left;
  while (root->right != NULL)
    root = root->right;
  }
  return root
```

Note: We always want min height BST so that Search time is reduced. To get that we perform Rotation.

Exit condition ←

```
if (root = NULL)
    return
if (root->left = NULL && root->right = NULL)
    for root
```

AVL → Special type of BST. follows left<node<right + tree must be balanced

height of left subtree − height of right subtree ≤ 1 ← within this range

(if tree becomes unbalanced → rotation)

Benifit → (faster Searching)

height → how far we can go down from node to leaf.



height of 2 → 0
5 → 1
10 → 2

Balance factor = leftheight − right height

↓

is the leftside heavier or Rightside

−1, 0, +1 ✓

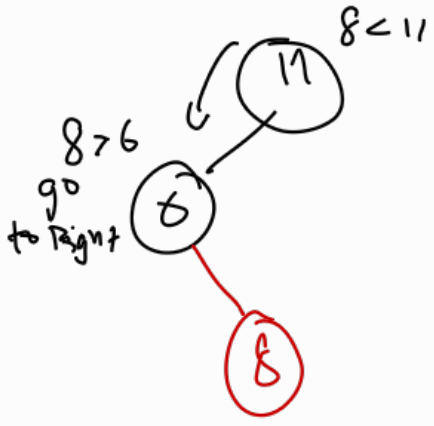
anything else is unbalanced

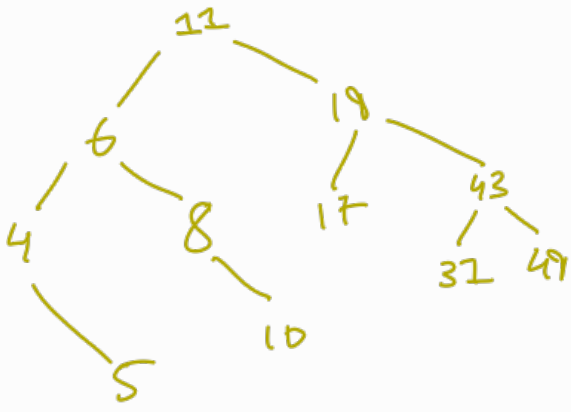
To fix unbalanced → rotation.

(not included in mid 2)

create Binary tree from left to right

11, 6, 8, 19, 4, 10, 5, 17, 43, 49, 31





Recursive Insertion

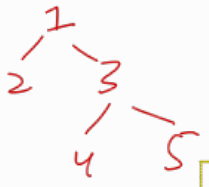
```
void insert (node* &root, int key)
{
    if (root == NULL)
        root = new node (key)
        return

    if (key < root->data)
        insert (root->left, key);

    else
        root -> right = insert (root->right, key)

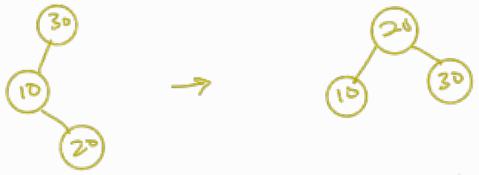
    return root;
}
```

(Morris Inorder traversal)



Inorder:: 2 1 4 3 5

Reducing height of BST → AVL (By applying Rotation)



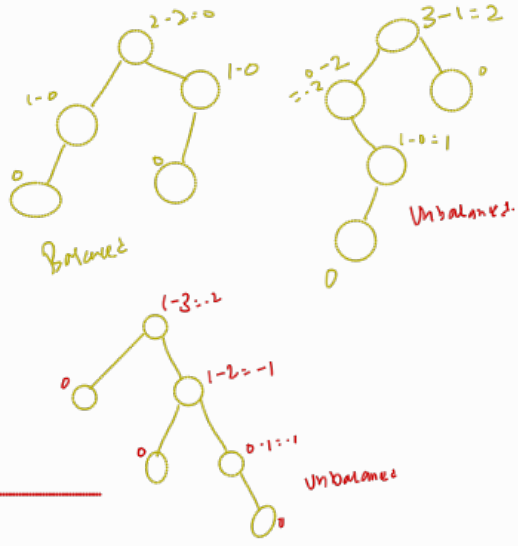
Balanced

→ we calculate Balance factor of each node = height of left subtree − height of right subtree

Bf = hL − hR → possible Range of values {−1, 0, 1}

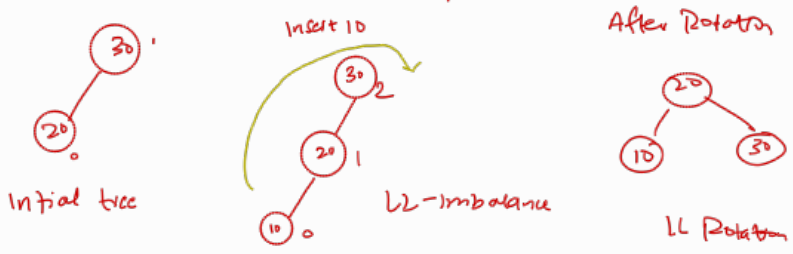
↙ If Bf has another value than these then the tree is unbalanced.

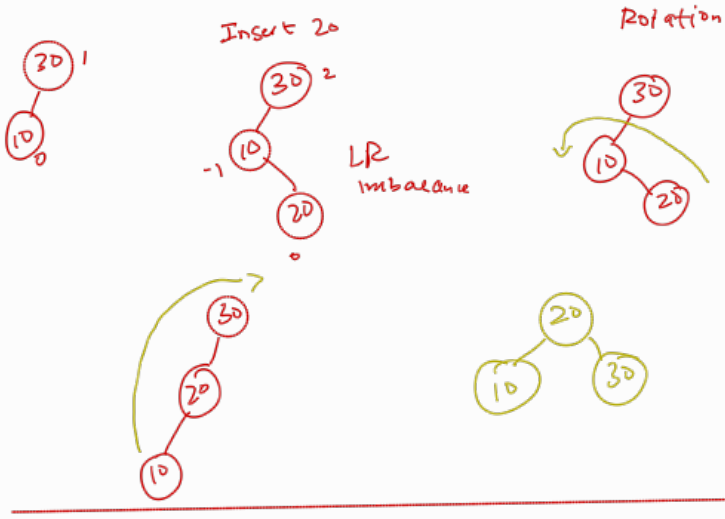
Balanced values



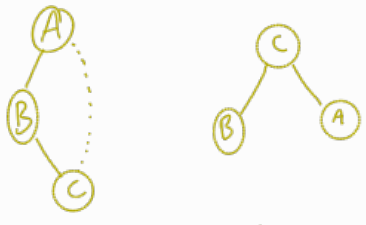
height → Longest path.

→ To Balance a tree Rotations are performed

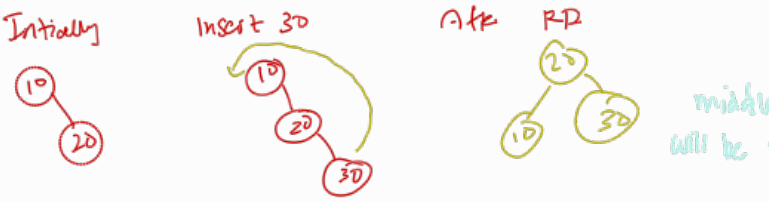




LR Rotation in Single step:



(Last node → Root)



middle element will be the root.



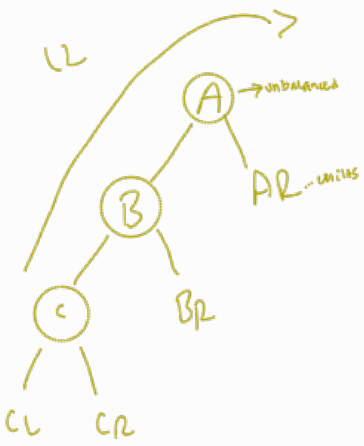
LL, RR → Single Rotation

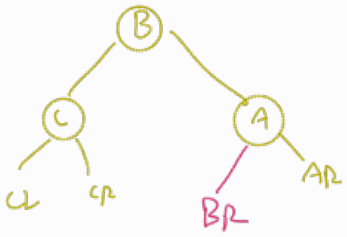
LR, RL → double rotations

Rotation are always done within 3 nodes

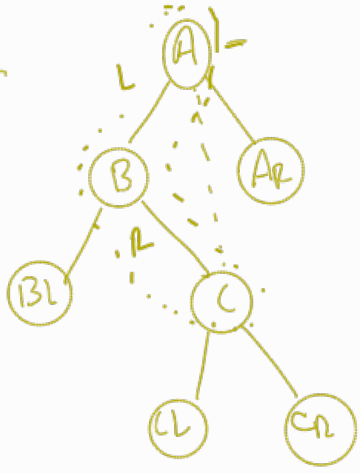


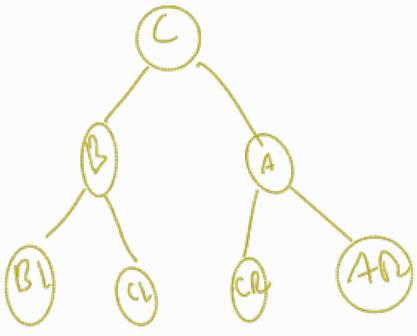
Special case

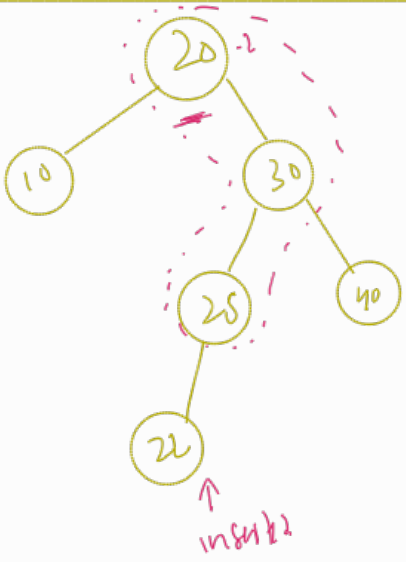




LR





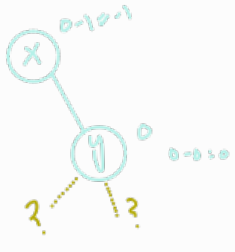


RL

From unbalanced node only take 2 steps

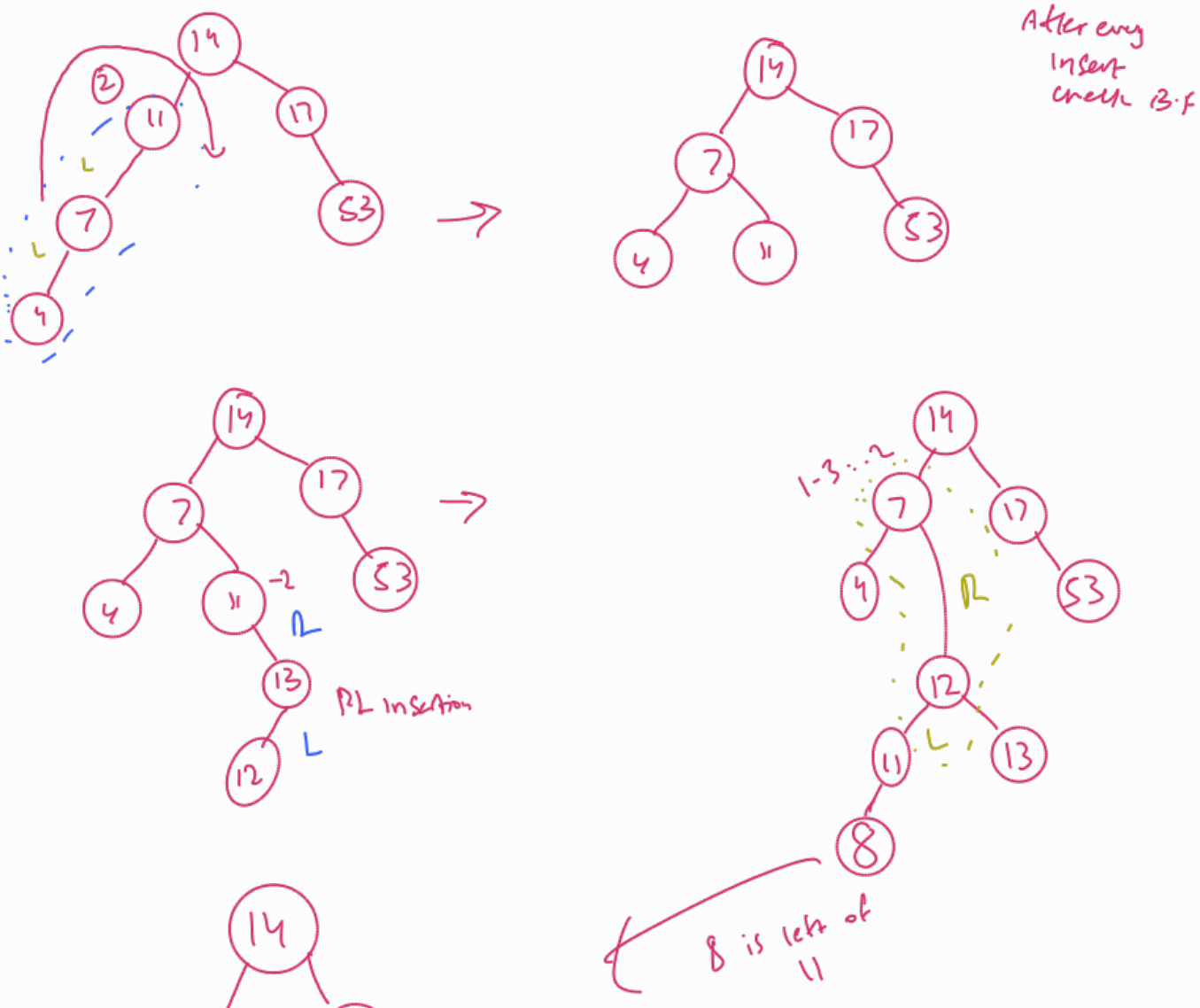
→ After every insertion find out the balance factor.
→ Balance factor is calculated from each node.

→ height of both Right / left Subtree is longest path until reaching leaf node

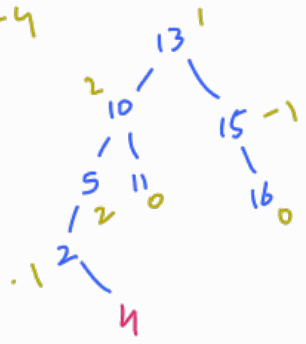


Construct AVL tree by inserting the following data:

14, 17, 11, 7, 53, 4, 13, 12, 8, 60, 19, 16, 20



Insert 4



when multiple nodes become unbalanced at same time during insertion, always fix the lowest (deepest first)



fix: Both 5, 10 at same time





LR

Insert: 6



Note:: After every Rotation check the B.f

Delete 5

Inorder:: 1 2 3 4 5 6 7 8 9 10 11 12

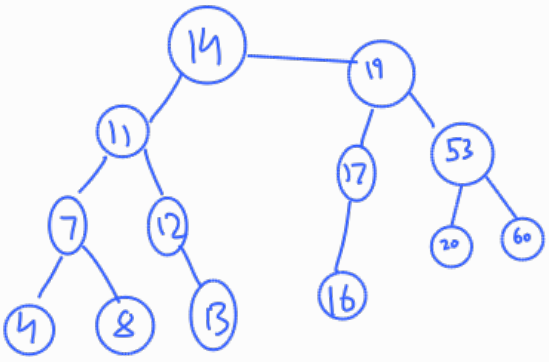
Replacing 5 with 4.



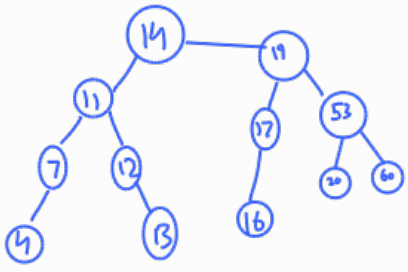


(Deletion in AVL tree)

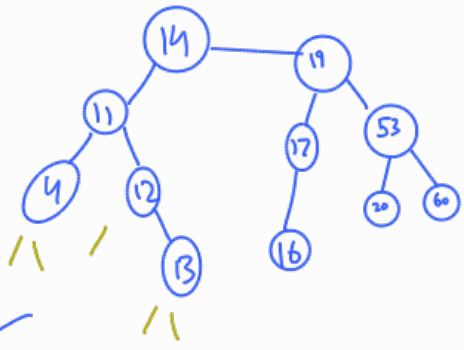
8, 7, 11, 14, 17 ← delete this



After deleting check out the Balance factor of each node

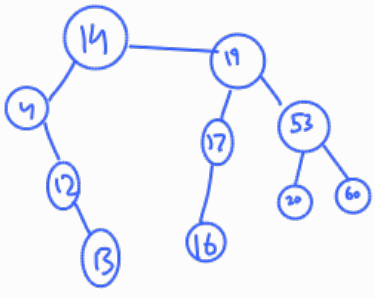


→ 7 deleting



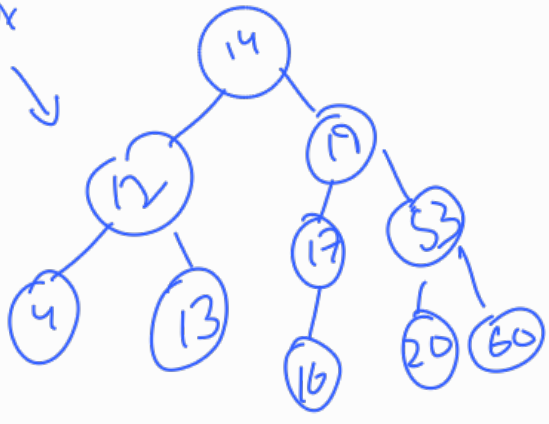
deleting 11

Replace 7 by either its left / Right child



Replace 11 by inorder predecessor/ postdecessor. → 4 11 12 13 ..

Bring 4 out



★ leaf node → just delete

delete 17



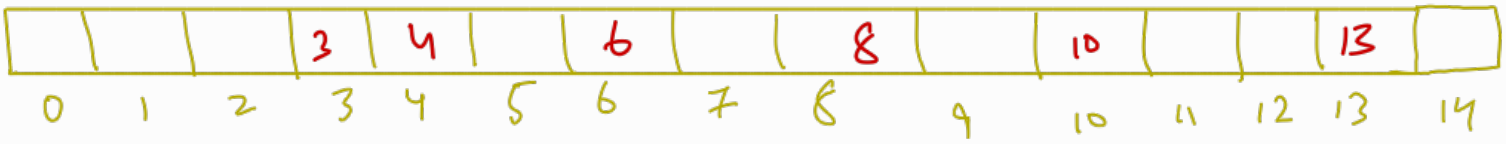
only 1 child replace it

→ 16

In deletion → always perform Rotation on the heavier side

Hashing → for Searching: Takes O(1) → constant time only.

Keys → 8, 3, 13, 6, 4, 10



Take the value and store it as the Index.

Key=10 → directly go to 10 and see if 10 is present.

Problem: Alot of space is wasted in Array



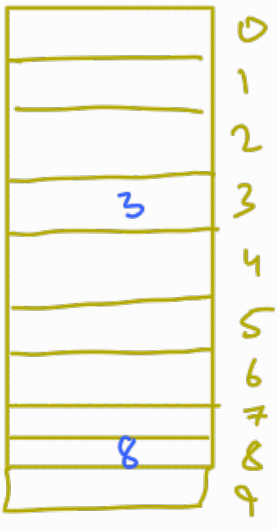
h(x) = x → hash function that maps the key elements to hash table

modifying function to use efficient space

h(x) = x % 10 → remainder

↓

Size of hash table (e.g.)

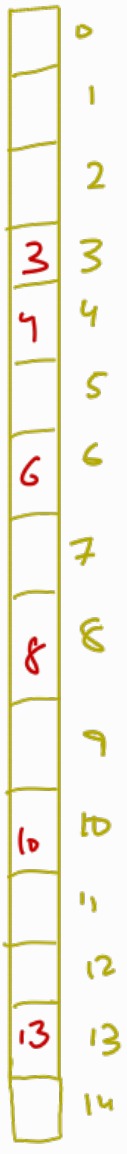


8 % 10 → 10 | 8 | 0 | 8

13 % 10 → 10 | 13 | 1 | 3

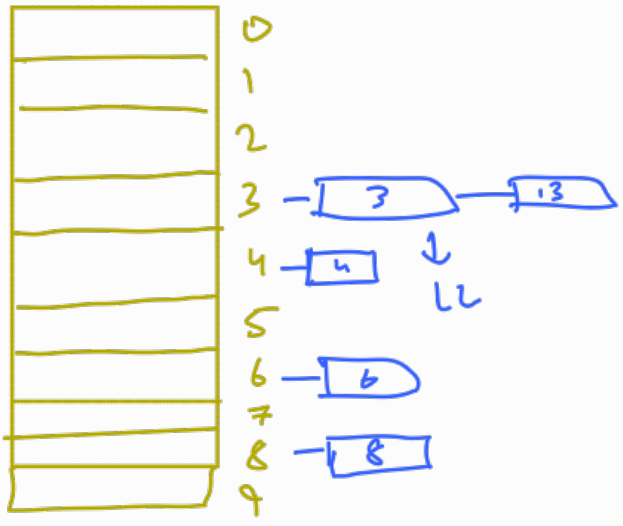
Both 3 and 13 are colliding | collision at index 3

Hashtable



(Avoiding collision)

1) Chaining: → using nodes



key: 13

13 % 10 → 3

goto 3 index But Still we need to Search the LL to find 13.

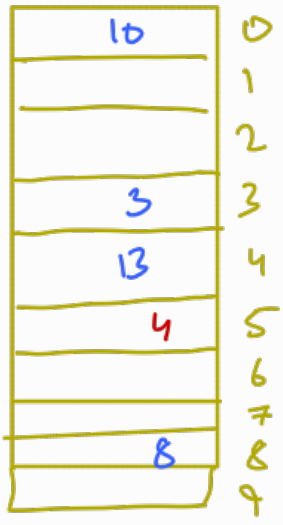
Another method:

Linear probing:

comes under open addressing

uses arrays

Where collision happens Just Store that element in next free Space



Search for

key: 4

4 % 10

→ (4)

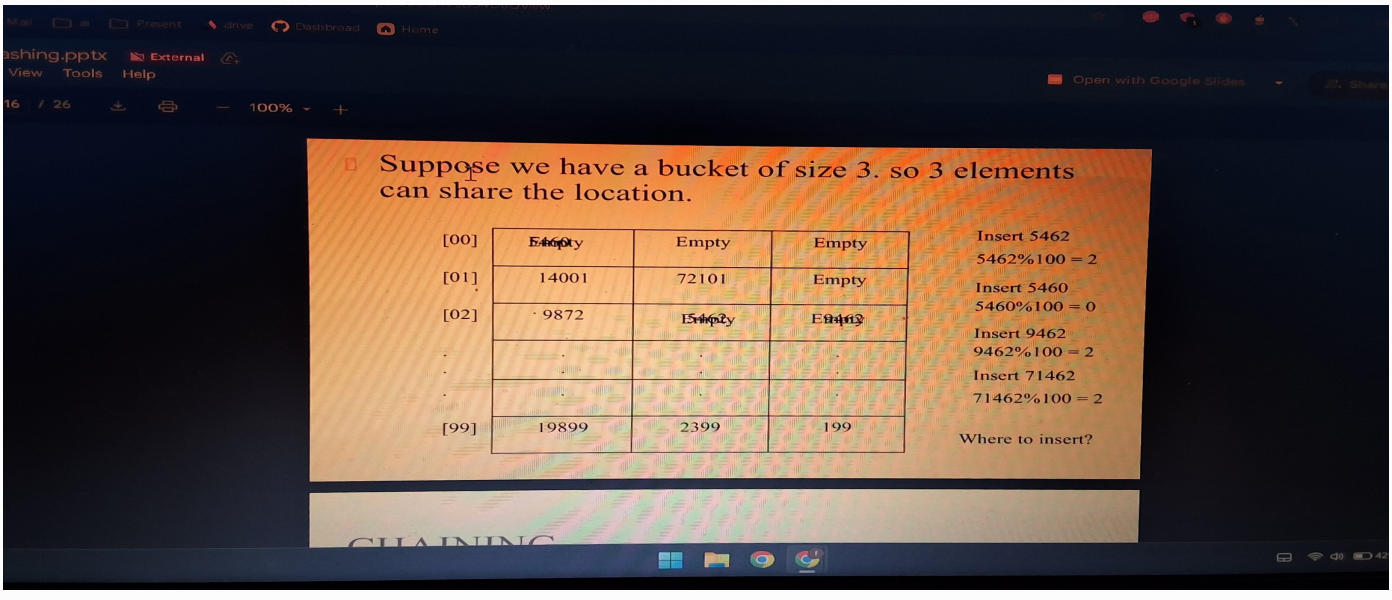
goto 4 index if not found goto next location. if not found again go to next location, if found empty Space say element not present.

Problem: Clustering of elements

Linear probing place the number at the nearest empty slot.

circular wrap around in Linear probing ??

Avoiding collision: Another method → Bucket comes under Seperate Chaining
uses L.L ←



common hashing functions:

Index -- key % Table size → preferably prime no

key:
↓
label used to find the LOL
(Same as data)

Hashfunction:
↓
this key into a Index no.

X.1.100

(Implementation)

```
class node {          // LL
    int key
    node * next

    node (int k) {
        key = k;
        next = nullptr;
    }
```

```
class Hashtable {
    int tableSize;
    node ** table;

    int indexval(int key) {
        return key % tableSize;
    }
Hashtable(int Size) {                    // initialization
    this -> tablesize = Size;
    table = new node *[tablesize];
        for (int i=0; i< tablesize; i++)
            table[i] = nullptr;
};
```

node * → points to another node

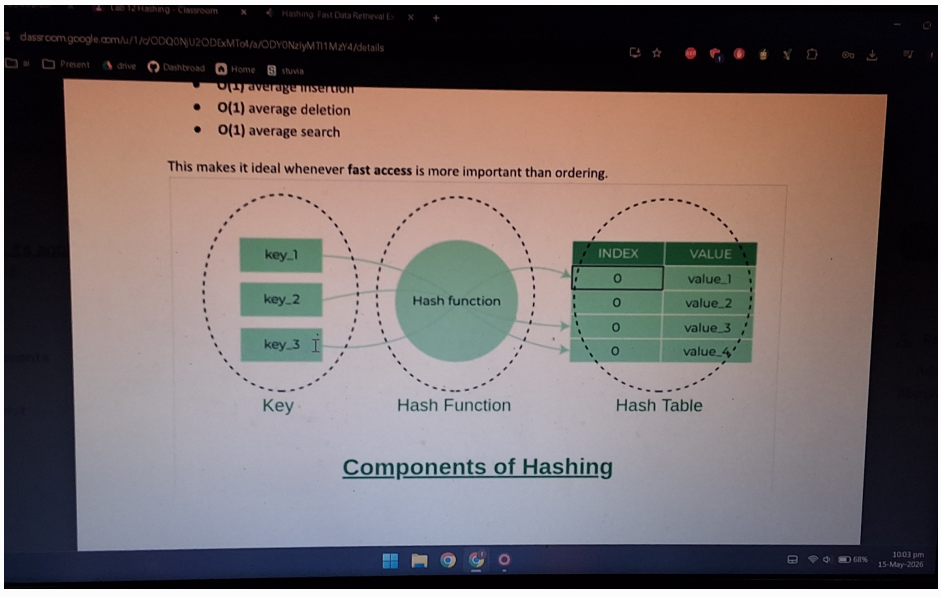
node ** t → pointer to a chain of nodes that are not connected

t = new node *[Size] → creates nodes that have no links

t → □ □ □ □

```
new -> next = table[index]      // So first point to where our head is pointing to then we point our new itself to head
table[index] = new
```

```
node *ptr = head    // Same as node * temp = table[index]
```



Load factor →

$$\frac{\text{no of entries}}{\text{table size}}$$

Advantages → flexibility (can hash almost anything) → username ID, profile

→ Speed O(1)

→ e.g. Bank acct login

Disadvantages

→ collision, clustering, Rehashing (Since size of hash table is fixed, you have to create a new table to store new items)

```
for(int i=0; i<size; i++) → L.P
    int trailnext = (hashIndex + i) % size //check for collisions
```

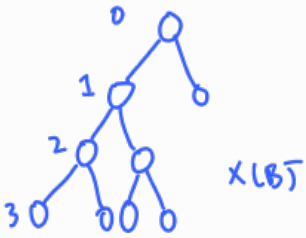
Q.P ← `int trailnext = (hashIndex + (i*i)) % size`

```
#define MAX 10 → Replaces MAX with 10
int arr[MAX];
```

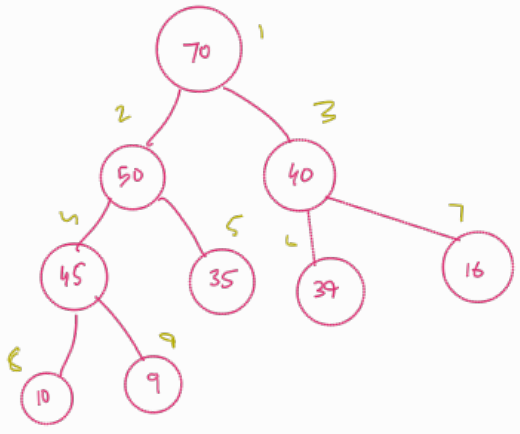
Heap → Tree Based data Structure / almost complete Binary tree

Max heap

Min heap



Max heap
every parent node is or equal greater then its child node.



→ Almost complete Binary tree only at the Last level is filled at left most.

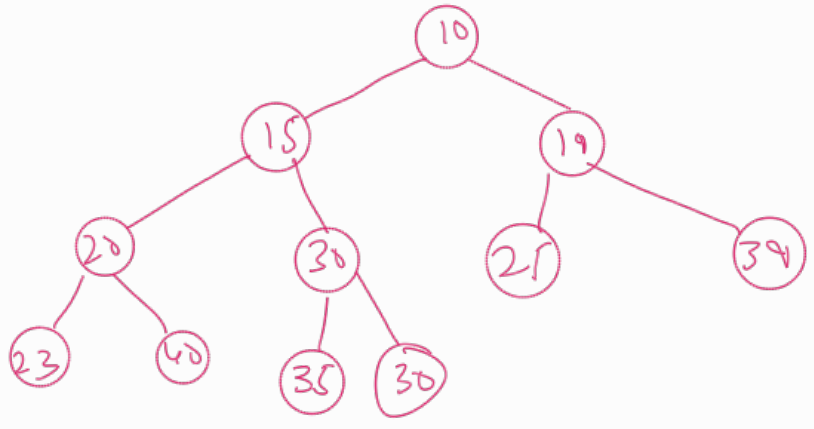
Array representation : writing level By level from left to right

70	50	40	45	35	39	16	10	9
1	2	3	4	5	6	7	8	9

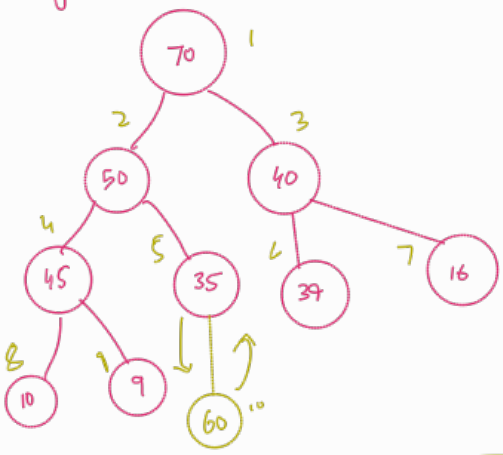
In max heap the root node will be the largest element.

Min heap

Parent node will be less then or equal to child node

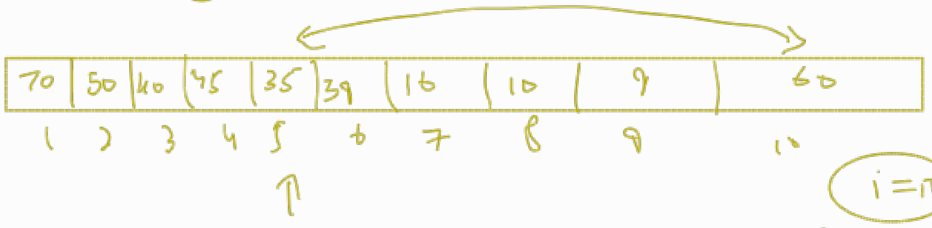


Inserting in max heap → Data is Inserted in leaf node |



insert (60)

must satisfy complete Binary tree property and max heap property.



i = 10

finding parent

Replace parent with 60.

again check parent

$\lfloor \frac{5}{2} \rfloor = 2.5 = 2$

Now Replace 50 with 60

Replace till we reach the Root node ||

→ Insert, find the parent, if greater than parent, parent becomes child keep on doing until Root node.

Parent Index - i

Right child → 2 * i + 2

Left child → 2 * i + 1

Child Index - i

Parent $\lfloor \frac{i-1}{2} \rfloor$

```
class Maxheap {
    int * arr;
    int size;   // no of elements in heap → 0 at start
    int total   // total size of array → 6
};   Assign values using const.   arr = new int[n]
                                   total = n
```

```
void insert (int val)
{   if (size == totalsize)
        cout<< "cannot insert";

    arr[size] = value;      // insert at last
    int index = size        //initially size = 0.
    size++

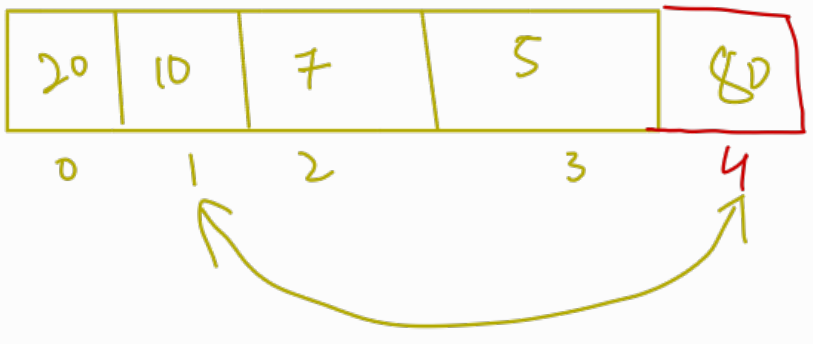
// comparing with parent
    while(index > 0 && arr[index-1]/2 < arr[index])
    {
        swap(arr[index], arr[index-2]/2)
              ↓ child          ↓ parent.

        Because child > parent

        index = (index-1)/2

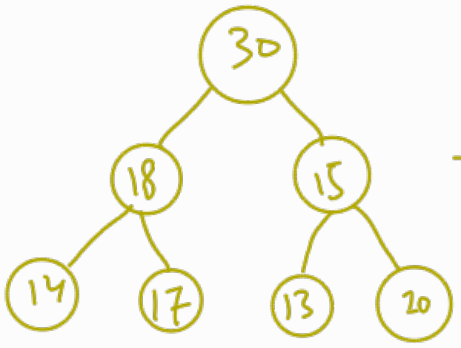
    }
```

Inserted into the heap



index = 4

(Delete a node) → only top node



→



Replace with Last node to maintain BST property



find child

$2 \times i + 1$

$2 \times i + 2$

convert to max heap

$2 \times 0 + 1 = 1$
$2 \times 0 + 2 = 2$] indexes → greater of them will Replace 10.

Move next Index

T.C → Logn

```
Void delete()
{ if (size==0)
      cout<< "no elements";
  cout<< arr[0] << " deleted root"
  arr[0] = arr[Size-1];
  Size--; //no of elements in heap decrease
  if (Size == 0)
      return
```

```
Void heapify(int index)
{
  int largest = index;
  int left = 2*index+1;
  int Right = 2*index+2;
  largest will store the index at which the Biggest node is stored
  if (left<size && arr[left]>arr[largest]
      largest = left;
  what if left index doesn't even exist??
  //Same for right
  if (largest != index)
      Swap(arr[index], arr[largest])
      Heapify(Largest)
```

meaning that it stores above changed the value of largest

to check again for Largest nodes

Bulid heap ⟶ nLogn

Child = i

Parent = $\left\lfloor \frac{i-1}{2} \right\rfloor$

0	1	2	3	4	5	6	7	8	9
14	15	12	22	17	20	18	24	23	30

15 ↗ 14

Start from here

$\left(\frac{1-1}{2}\right) = 0$

if child is greater than parent → Swap.

lowering to Max heap

Start at index → 1 ← Step up → Child compare with parent. (Top → Bottom)

Start at ← Step down → Parent compare with child (Bottom → top)

Index $\left[\frac{n}{2} - 1\right]$

```
Void BuildHeap (int arr[], int size)
{
    for (int i = n/2 - 1; i >= 0; i--)   // Starting from that node which has children
    {
        Heapify (arr, i, size)
    }
}
```

Shutdown

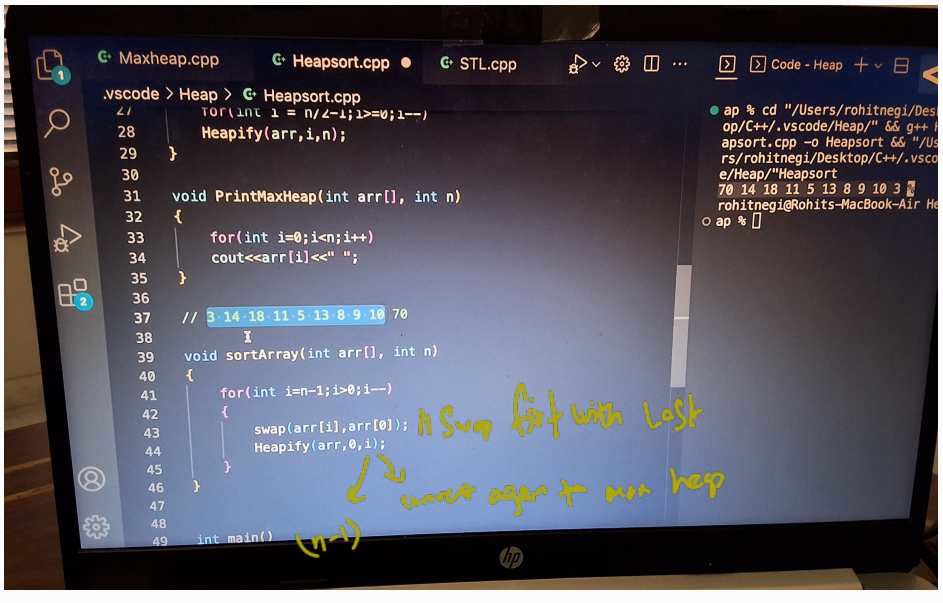
Heap Sort



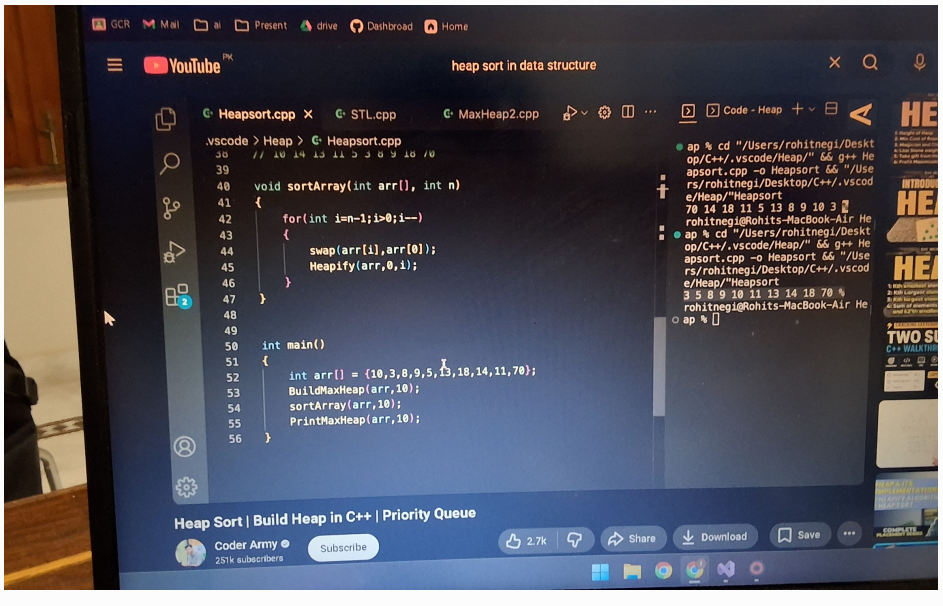
Buildheap → (heapify)

Take a array as input and convert to max heap delete top root element, and replace by last element, apply heapify and again remove the top root element.

1) Replace first element with last element
2) Decrease size of array by 1.
3) Apply heapify



1) convert arr to Maxheap → BuildMax heap
2) BuildMaxheap → Sorted array



graph is a data structure. → a way to store and organize data.

Trees are special cases of graphs.

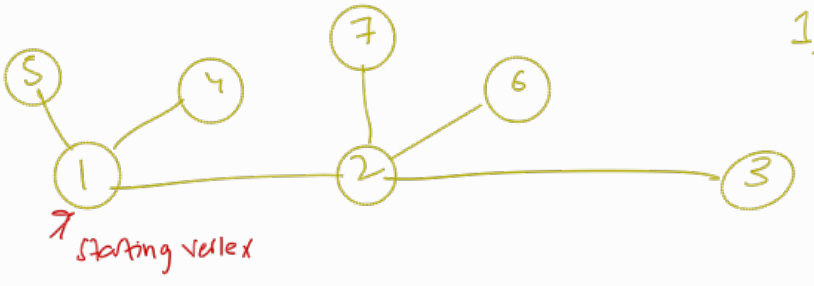
Adjacent nodes → 2 nodes are adjacent if they are connected by an edge



complete graph → in which every vertex is directly connected to every other vertex.

graph implementation
- → array based
- → LL

Traversal:



1) Visiting a vertex
2) exploration of vertex.
↓
visiting all the particular vertexes is called exploration

BFS: → cover level by level 1, (5, 4, 2), 7, 6, 3
(expand; nothing connected)

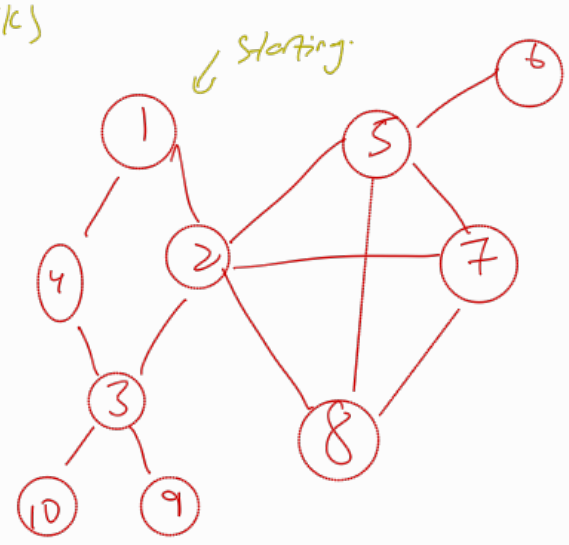
DFS: → one direction till the very end and then Back track.

1 → 2 → 3 → (Back track) 6 → 7 → 4 → 5

BFS :: Queue

(Take any vertex v like)

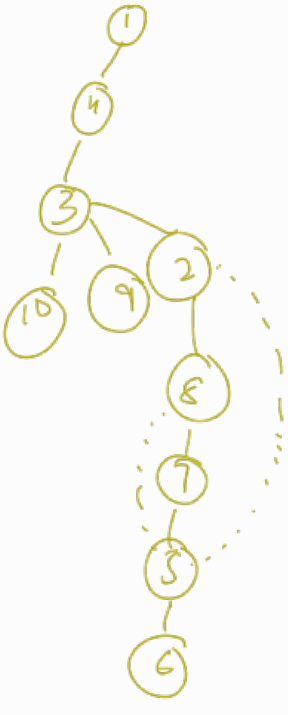
x	4	2	3	5	8	7	10	9	6

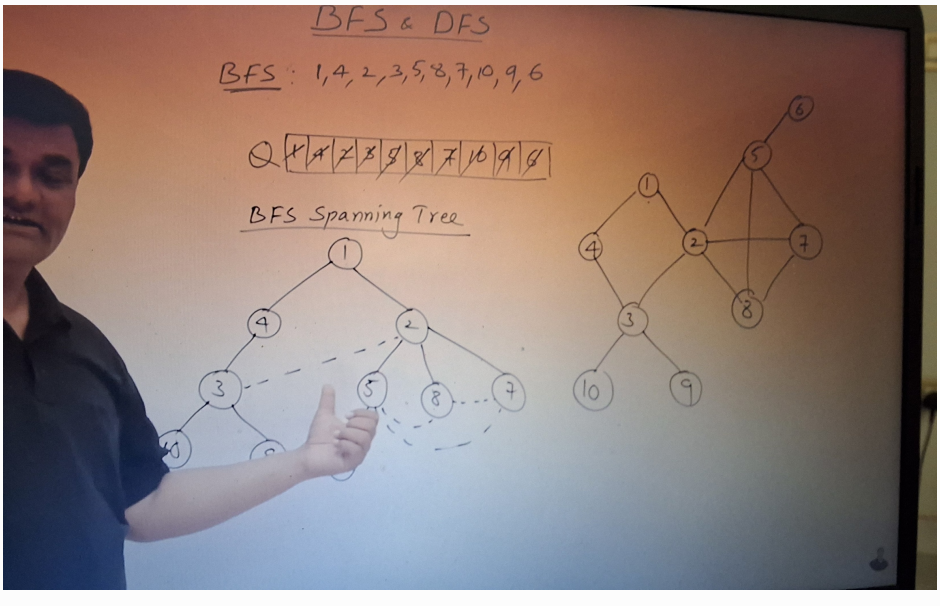


DFS

visited 1, 4, 3



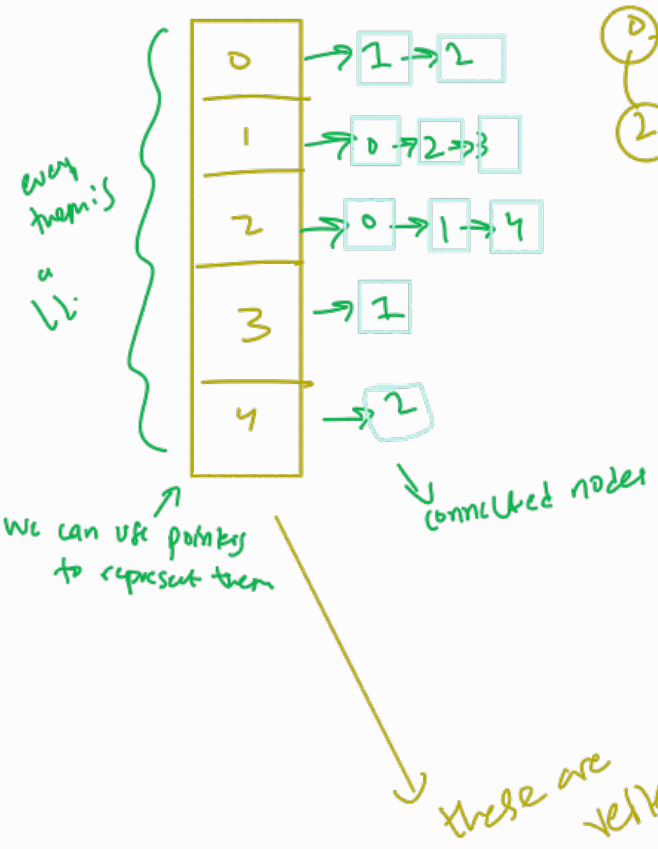


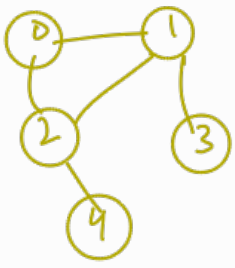


graph representation

→ Adjacency list :. mark nodes with its neighbours.

→ Adjacency matrix



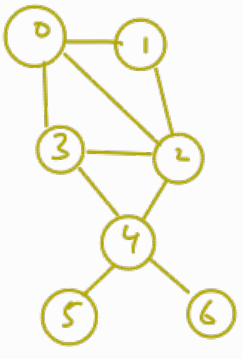


	0	1	2	3	4
0	0	1	1	0	0
1	1	0	1	1	0
2	1	1	0	0	1
3	0	1	0	0	0
4	0	0	1	0	0

these are vertices

if Sparse graph use list

dense use matrix

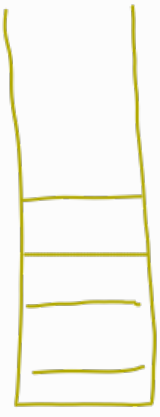


DFS

visited: 0 3 4 6 5 2 1

Stack:





Start by putting any 1 vertices on top of a stack. and add that to visited list

Now pop that vertex and add its child in stack

if a node is already been visited don't push it into stack

only mark nodes visited after they have been popped from stack

(DFS using code)

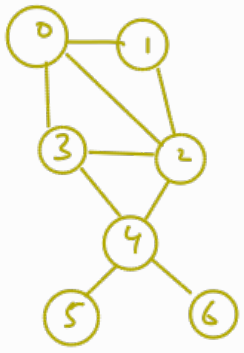
```
int visited [7] = {0, 0, 0, 0, 0, 0, 0}
int a [7][7] = {{0, 0, 0, 0, 0, 0, 0,
                 0, 0, 0, 0, 0, 0, 0,
                 0, 0, 0, 0, 0, 0, 0,
                 0, 0, 0, 0, 0, 0, 0
                 .
                 .
                 .
                 1
                                    };};
```

```
void DFS (int i) {                    // function that asks user which
                                         node to visit first
    int j
    cout<< i;
    visited [i] = 1;
for (int j=0; j<7; j++)
  {   if ( a[i][j] ==1 && ! visited [j] ) {
                DFS (j);
        }
    }
```

#Include Stack

datastructure <datatype> variablename

1) Started from 0, Insert in queue and visited.
2) Delete from queue and add its children to Both.



Visited: 0 1 2 3 4 5 6

Queue: ~~0~~ ~~1~~ ~~2~~ ~~3~~ ~~4~~ ~~5~~ ~~6~~

Huffman coding → compression

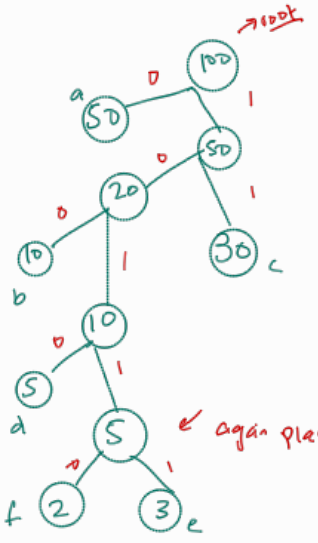
In ASCII
(0-127) → 7bits

a = 50
b = 10
c = 30
d = 5
e = 3
f = 2

} occurances

Message Length = 100 characters.

= 7x100 = 700bits it will take to represent.

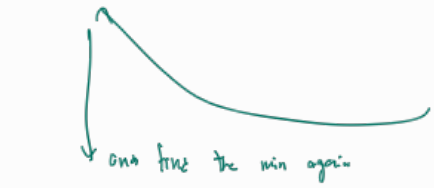


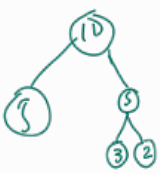
a = 0 (a represented in 1 bit only that is 0) 50x1
b = 100 (3x10)
c = 11 (2x30)
d = 1010 (5x4)
e = 10111 (5x3)
f = 10110 (5x2)

again place in box

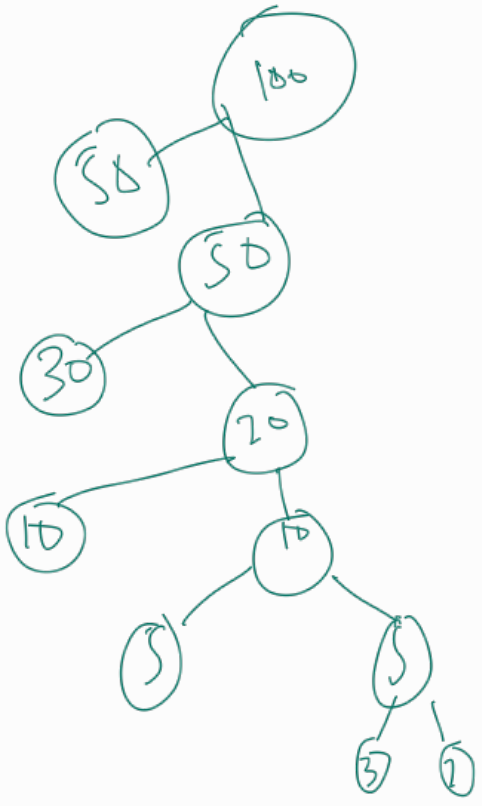
Smaller element left side, larger element Right side

Place 0 at left and 1 at Right





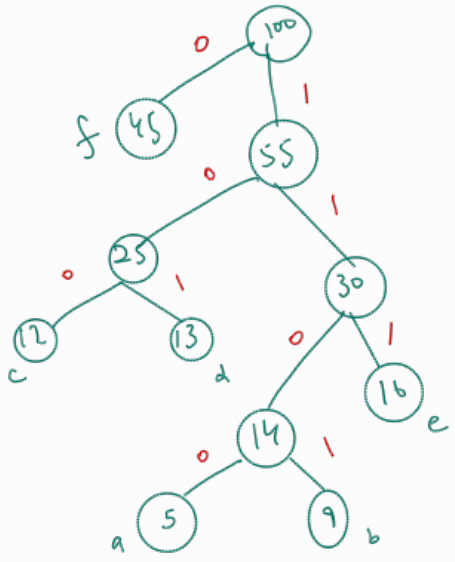




if total root node is
Same as 0+1+(1+1...
then we have
is correct.

Right child => 2
left child => 0

~~a: 5~~
~~b: 9~~
~~c: 12~~
~~d: 13~~
~~e: 16~~
~~f: 45~~
~~14~~
~~25~~
~~30~~



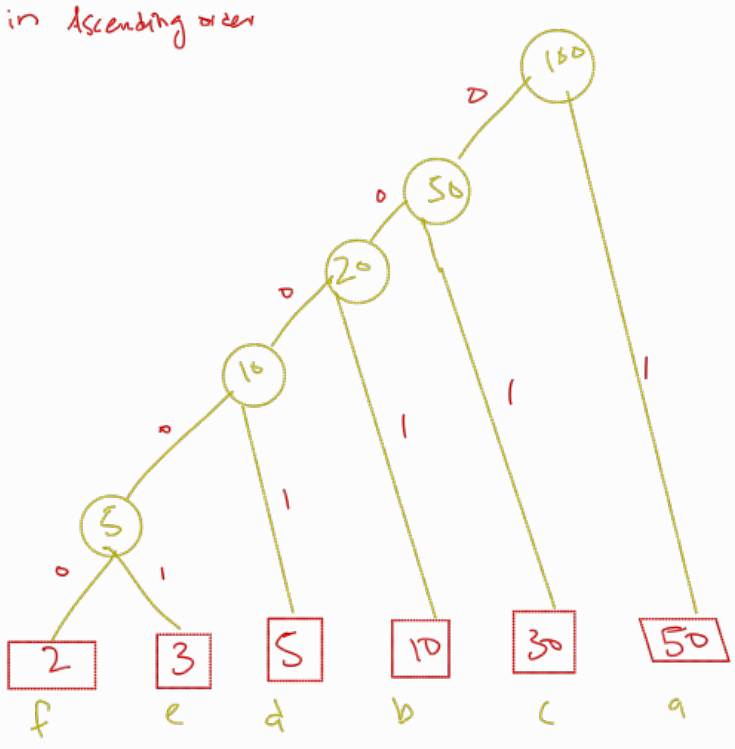
a: 1100 20
b: 1101 36
c: 100 36
d: 101 39
e: 111 48
f: 0 45

324

---

a: 50
b: 10
c: 30
d: 5
e: 3
f: 2

unix in ascending order



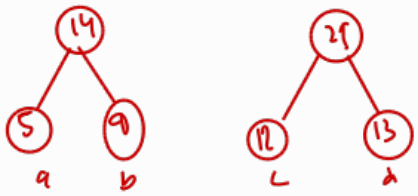
a: 1 → 1x50
b: 001 → 3x10
⋮

1) first take the smallest 2 and Build tree

2) then find the next 2 ones with least freq and Build tree

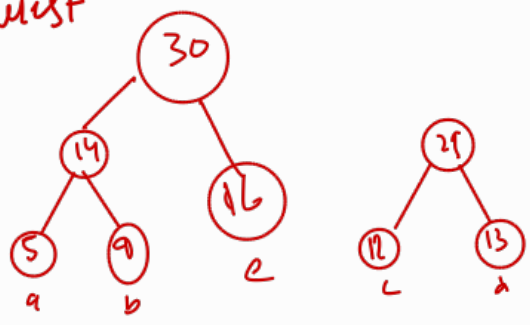
3) Rearrange freq in Ascending

~~a~~: 5
~~b~~: 9
~~c~~: 12
~~d~~: 13
e: 16
f: 45

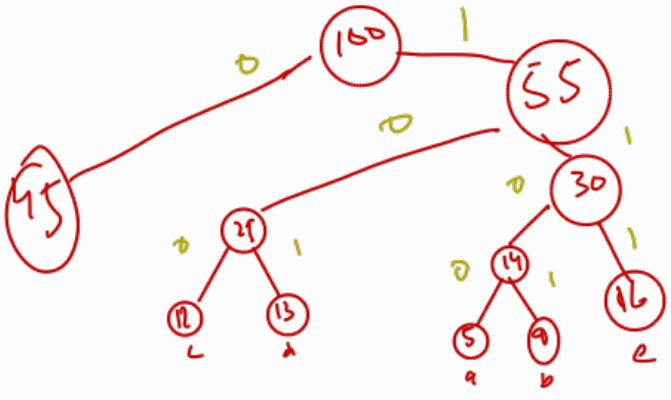


left 14, 25, 16, 45

Smallest



25, 30, 45



a: 4 x 5
b: 4 x 9
c: 3 x 12
d: 3 x 13
e: 3 x 16
f: 1 x 45

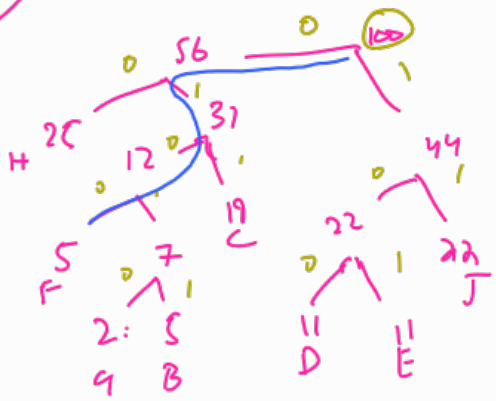
→ 224

MM freq will always be on left side

Sort the freq

G:2 B:5 F:5 D:11 E:11 C:19 J:22 H:25

↳ updated F:5 7 D:11 ...



now combine the smallest.

22 J:22 H:25 31

25 31 44

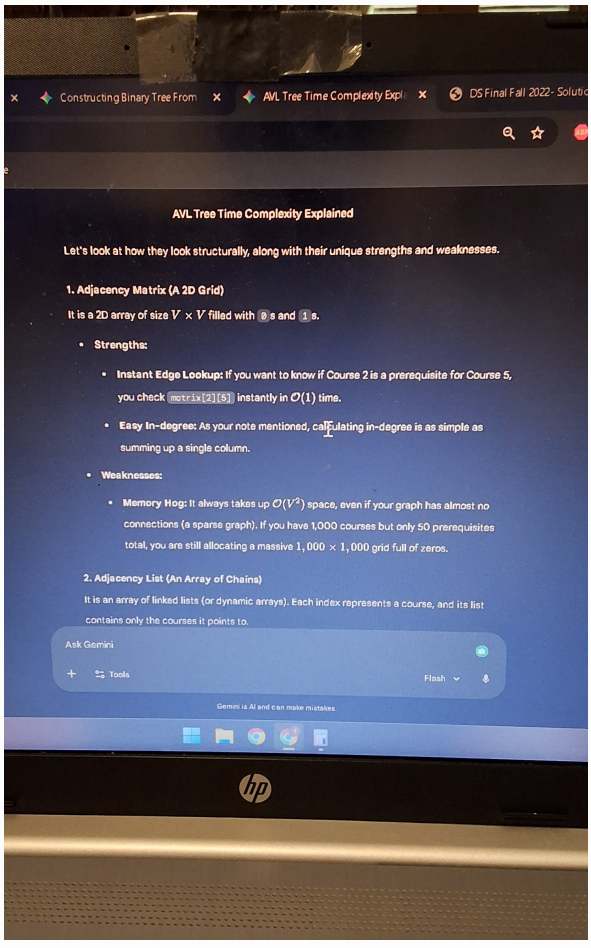
56 44

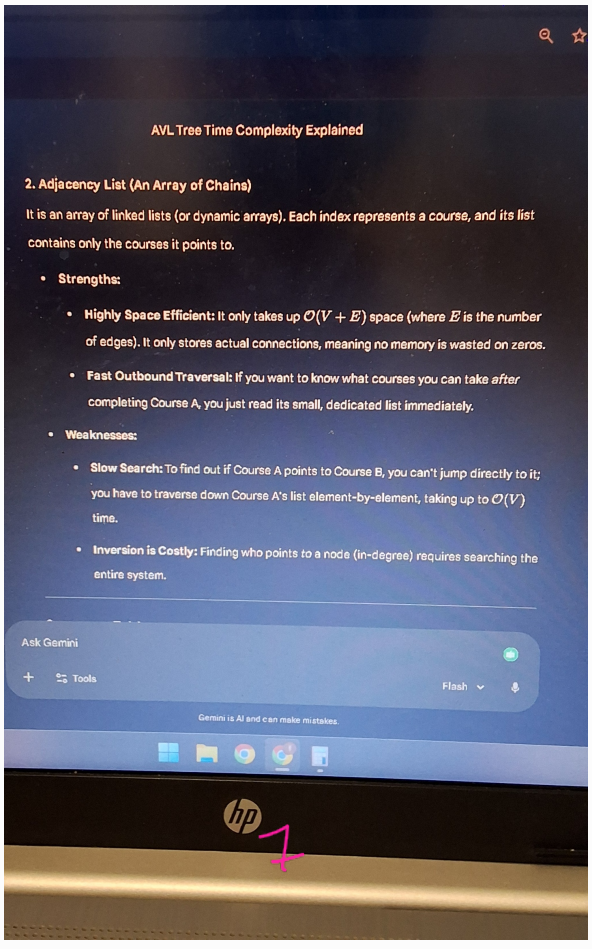
Sum 224

8 bits = 1 byte

(5x2) + (5x5) + (3x11)

(2x3) + (19x3) + (4x5) + (3x11) + (22x2) = 272 bits


Constructing Binary Tree From
AVL Tree Time Complexity Expl
DS Final Fall 2022- Solutio
AVL Tree Time Complexity Explained
Let's look at how they look structurally, along with their unique strengths and weaknesses.
1. Adjacency Matrix (A 2D Grid)
It is a 2D array of size $V \times V$ filled with 0s and 1s.
Strengths:
Instant Edge Lookup: If you want to know if Course 2 is a prerequisite for Course 5, you check matrix[2][5] instantly in $O(1)$ time.
Easy In-degree: As your note mentioned, calculating in-degree is as simple as summing up a single column.
Weaknesses:
Memory Hog: It always takes up $O(V^2)$ space, even if your graph has almost no connections (a sparse graph). If you have 1,000 courses but only 50 prerequisites total, you are still allocating a massive $1,000 \times 1,000$ grid full of zeros.
2. Adjacency List (An Array of Chains)
It is an array of linked lists (or dynamic arrays). Each index represents a course, and its list contains only the courses it points to.
Ask Gemini
Tools
Flash
Gemini is AI and can make mistakes.

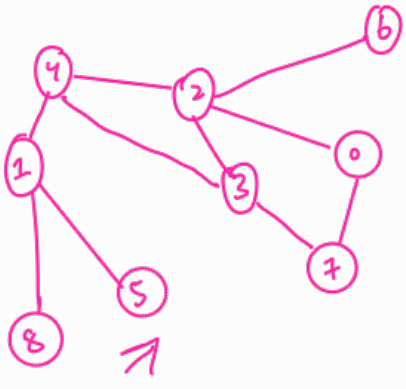


DFS

visited: 5 1 4 2 0 6 7

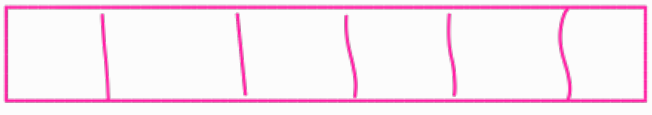
Stack:





BFS

visited: 5 1 4 8 2 3 0 6 7



front (dequeue)

rear (insert)

(5 1 4 3 7)